HOOX · PYNE · DOCUMENTATION

API CLIENTS · BACKEND OPERATORS

PRO API MANUAL

Flask Pro API, auth, /run contract, preview, backtest

Open-source Pine Script evaluation framework

CONTENTS

01 Overview

02 App Lifecycle

03 Auth And Keys

04 Run

05 Sessions

06 Data Plane

07 Stats

08 Runner

09 Optimize

10 Preview

11 Backtest

12 Chart Renderer

13 Runtime Bridge

14 Contract

OVERVIEW

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE
HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

---

title: "Pro API" description: "Flask Pro API — /run with mode=auto, /optimize strategy search, plot_meta/fill, alerts + L2 webhooks, preview, backtest, and shared evaluate contract."

PRO API

The PYNE **Pro API** is a Flask HTTP service that runs Pine scripts over caller-supplied OHLCV, returns plots / series / `plot_meta` (incl. `fill` bands) / strategy events / drawings / **alerts**, optional **L2 alert webhooks**, and gates thumbnail + backtest routes behind API keys. Default evaluate path is `mode=auto` (compile when safe, interpret on fallback). It is intentionally **not** the language server: different auth, scaling, and failure domains.

Abstract

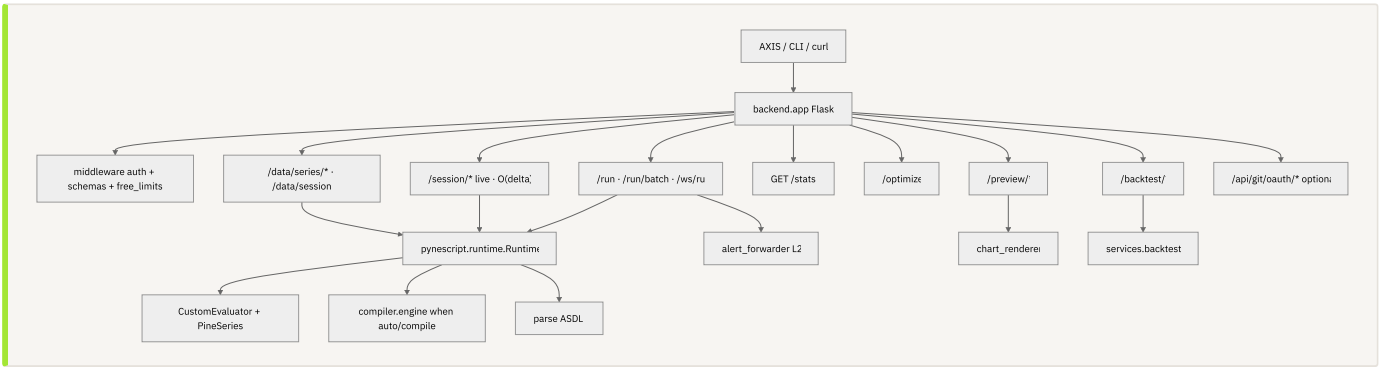
Where the LSP answers *static* editor questions, the Pro API answers *dynamic* evaluate questions:

```
POST JSON { script, data: OHLCV[], mode?, libraries?, webhook_url?, … }
  → pynescript.runtime.Runtime.run  (Pro API re-export; mode default: auto)
  → { plots, series, plot_meta, events, drawings, alerts,
      mode, auto_backend?, compile_fallback_reason?,
      script_id, run_id, alert_forward?, … }
```

Free-tier evaluate (`POST /run` , `POST /run/batch`) validates bodies with hand-rolled schemas, enforces a 5 MB body cap, and restricts CORS. Optional **bar/script/rate/concurrency** guards (`backend/middleware/free_limits.py`) are **off unless** `FREE_TIER_LIMITS` is `1` / `true` / `yes` / `on` . `GET /` and `GET /health` are the same unauthenticated readiness payload (`features.free_tier_limits` reports the switch). Optional **L2 webhooks** POST last-bar `alert()` / `alertcondition()` firings to `webhook_url` or env `ALERT_WEBHOOK_URL` (see Alerts). Pro routes under `/preview/*` and `/backtest/*` use `track_usage` (Bearer / ApiKey / query key). Chart PNGs are matplotlib Agg renders encoded as base64.

The same **evaluate contract** is mirrored by **pyne-worker** so AXIS and HOOX can swap hosts without redesigning payloads — see contract.

Conceptual model



Interface surface

Method	Path	Auth	Role
`GET`	`/` · `/health`	none	Health + endpoint map + compile cache section
`GET`	`/stats`	none	Process runtime counters, latencies, gauges (docs)
`POST`	`/run`	none (free)	Single-script evaluate (`mode` default `auto` ; `libraries[]` ; `from_bar` / `to_bar` ; alerts + optional webhooks)
`POST`	`/run/batch`	none (free)	≤8 scripts, shared OHLCV
`POST`	`/session`	none (free)	Live session — O(1) ticks, O(delta) appends (docs)
`GET` / `POST` / `DELETE`	`/session/<id>` · `…/tick` · `…/append`	none (free)	Advance / inspect / release a live session
`GET` / `POST`	`/data/series` · `/data/series/bars` · `/data/series/provide` · `/data/series/fetch`	**API key**	Engine-owned OHLCV store (docs)
`POST`	`/data/session` · `/data/session/<id>/tick`	**API key**	Live session bound to a stored series
`POST`	`/scripts` · `/cron/*`	optional runner	Hosted scripts + bar-close — **off** unless `PYNE_RUNNER=1` (runner)
`POST`	`/optimize`	none (free)	Strategy `input.*` search — N interpret Runtime runs (docs)
`WS`	`/ws/run`	none (free)	Same evaluate contract over JSON frames (`flask-sock` optional)
`POST`	`/preview/chart`	API key + usage	Line / OHLCV PNG
`POST`	`/preview/indicator`	API key + usage	Expression series PNG
`POST`	`/backtest/quick`	API key + usage	Quick strategy metrics + equity PNG
`POST`	`/auth/create_key`	admin decorator	Mint `pyn_…` key (fail-closed `ADMIN_TOKEN`)
`GET`	`/auth/usage`	API key	Usage counters
`POST`	`/auth/validate`	none (body key)	Validate without consuming quota
`POST`	`/compile/prewarm`	none (free, rate-gated)	Product warm-compile — skip cold Numba JIT
`POST`	`/lsp/*`	none (free)	AXIS editor completion / hover / diagnostics
`POST`	`/api/git/oauth/device/*`	none (device	Optional GitHub/GitLab device OAuth for SPA git (blueprint)

Method	Path	Auth	Role
		flow)	

`/run` response highlights

Field	Role
`series` / `plots` / `plot_meta`	Bar values + display meta (color, linewidth, `kind`, `fill` band refs)
`events`	Strategy broker events
`drawings`	Drawing registry / compile `__drawings`
`alerts`	Structured `alert()` / `alertcondition()` firings
`mode` / `auto_backend`	Effective path (`interpret` | `compile`)
`compile_fallback_reason`	Why `auto` chose interpret (eligibility or compile error)
`alert_forward`	Present when a webhook URL was configured
`error_kind`	On failure: `parse` | `compile` | `runtime` | `data` | `order` | `mode`

Local runner: `make run` → `python -m backend.app` (default `127.0.0.1:5002`).

Tracks in this tab

1. App lifecycle — Flask app, CORS, limits, blueprints
2. Auth and keys — tiers, stores, decorators
3. Run · Live sessions · Data plane · Stats
4. Optimize · Preview · Backtest · Runner
5. Chart renderer
6. Runtime bridge — `Runtime` / series / evaluator glue
7. Contract — shared evaluate schema across Flask and workers

Internals (map)

Path	Role
`backend/app.py`	App object, `/run*`, `/session*`, `/data/session*`, `/health`, `/stats`, `/compile/prewarm`, `/auth*`, CORS
`backend/api/preview.py`	Preview + backtest blueprints
`backend/api/data_series.py`	Engine-owned OHLCV store routes
`backend/api/datafeed.py`	CCXT gateway + `WS /datafeed/watch`
`backend/datastore/*`	SQLite OHLCV store, session ↔ series binding
`backend/middleware/sessions.py`	Process-local live-session store (TTL + capacity)
`backend/middleware/window.py`	`apply_window` — response windowing
`backend/logging_setup.py`	`PYNE_DEBUG` level policy
`backend/api/git_oauth.py`	Optional device OAuth for SPA git
`backend/api/lsp_http.py`	Free AXIS editor LSP-HTTP
`backend/middleware/*`	Auth, schemas, free_limits, optional Redis/SQLite key stores
`src/pynescript/runtime/host.py`	Package SoT bar-loop (`mode=auto` , sessions, alerts pack)
`src/pynescript/stats.py`	Runtime counter / latency / gauge registry
`backend/runtime.py`	Compat re-export of the package host
`backend/alert_forwarder.py`	L2 HTTP webhook delivery
`backend/evaluator.py` / `backend/series.py`	Compat shims
`backend/services/*`	Charts + backtest simulation

See also

- Evaluate scripts (end user)
- Pro API usage guide
- Alerts
- Runtime hub
- LSP hub — editor path, not HTTP

APP LIFECYCLE

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE
HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

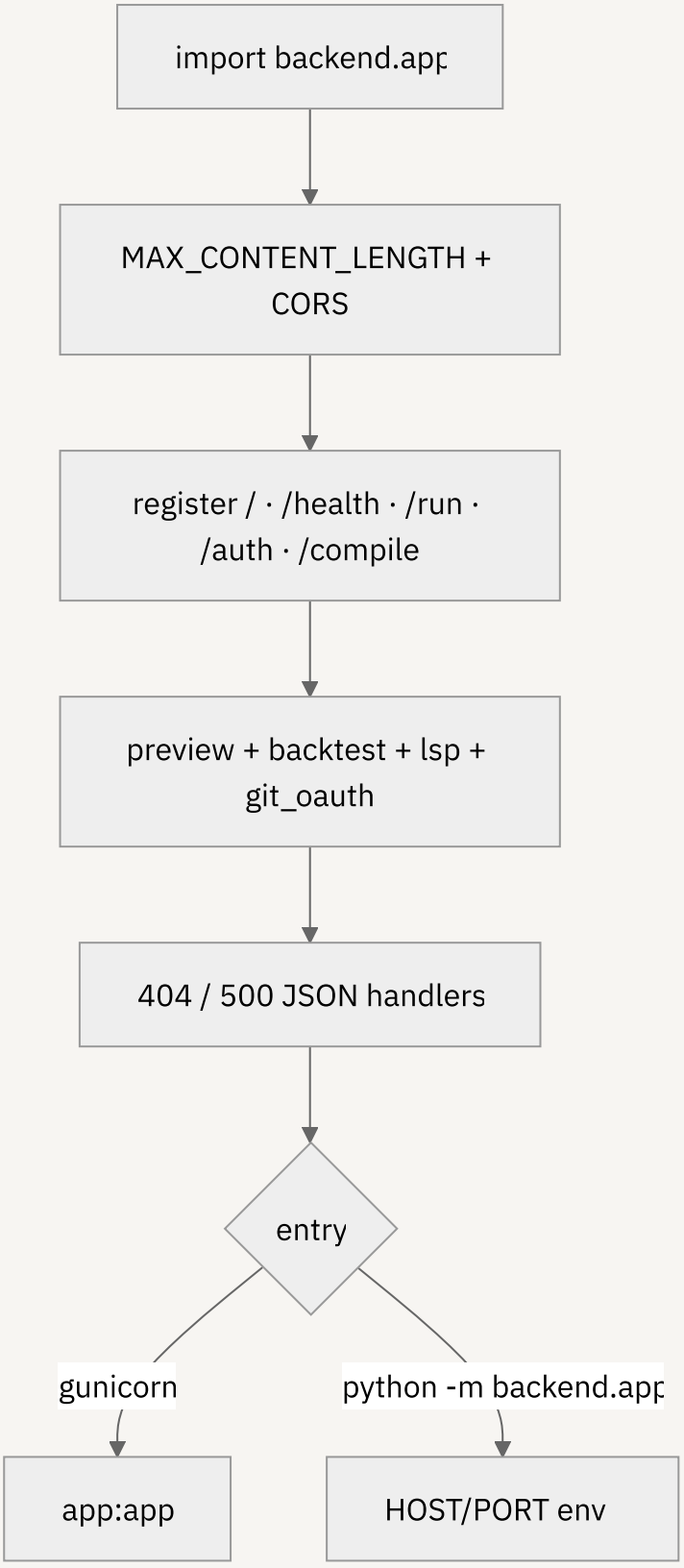
title: "App Lifecycle" description: "Flask application setup: MAX_CONTENT_LENGTH, CORS policy, blueprints, error handlers, and process entry."

APP LIFECYCLE

Abstract

`backend/app.py` constructs a single global `Flask` app, applies security-minded defaults (body size, CORS allowlist), registers free and Pro routes, and exposes a module `__main__` for local development. There is no multi-app factory pattern today — import `app` for WSGI (gunicorn) or run the module for the built-in server.

Conceptual model



Interface surface

Process entry

```
make run
# equivalent
python -m backend.app
```

Environment:

Variable	Default	Role
`HOST`	`127.0.0.1`	Bind address (localhost-first for dev)
`PORT`	`5002`	Listen port
`ALLOWED_ORIGINS`	`https://pynescript.ai`, `https://app.pynescript.ai`, localhost regex	CORS origins (comma-separated; regex allowed). `*` / `any` opens all. Code **always appends** localhost, private-LAN, and product (`hoox.sh` / `pynescript.ai` / `pynescript-axis.pages.dev`) regexes unless the value is `*`
`API_KEY_STORE`	`/data/api_keys.json`	JSON key-store path (`auth.APIKeyStore` default)
`ADMIN_TOKEN`	unset	Fail-closed admin minting (see auth). When set, hosted-runner writes also require it
`PYNE_RUNNER`	off	Optional hosted script registry + `/cron/run` (runner)
`PYNE_RUNNER_SCHEDULER`	off	In-process bar-close poll (requires `PYNE_RUNNER`)
`PYNE_RUNNER_POLL_SECONDS`	`60`	Scheduler interval
`PYNE_RUNNER_DB`	`/data/runner.db` or temp	SQLite registry path

`debug=False` always on the dev runner.

Health

`GET /` and **`GET /health`** are the same handler. AXIS Settings probes `/health`. Payload includes compile-cache diagnostics:

```
{
  "status": "healthy",
  "service": "pynescript-pro-api",
  "version": "1.0.0",
  "timestamp": 0,
  "websocket": true,
  "features": {
    "alerts": true,
    "alert_webhooks": true,
    "warm_compile": true,
    "default_run_mode": "auto",
    "script_runner": false,
    "script_runner_scheduler": false
  },
  "compile": {
    "has_numba": true,
    "disk_cache_enabled": true,
    "prewarm_enabled": true,
    "default_mode": "auto"
  },
  "endpoints": { "…": "…" }
}
```

`version` here is the **API service** label (`1.0.0`), not the `hoox-pyne` package version.

Hard limits

- **`MAX_CONTENT_LENGTH = 5 * 1024 * 1024`** — reject oversized bodies before JSON parse fills memory (audit 2026-07-05 / S1).
- Free compute is optionally gated by `backend/middleware/free_limits.py` when `FREE_TIER_LIMITS` is truthy (default **off**): max bars (`FREE_MAX_BARS`, 5000), script chars (`FREE_MAX_SCRIPT_CHARS`, 256 KiB), per-IP rate (`FREE_RATE_LIMIT` / `FREE_RATE_WINDOW_SEC`, 60 / 60s), concurrency (`FREE_MAX_CONCURRENT`, 4). Chart/`mock` data sources only. Production compose sets `FREE_TIER_LIMITS=1`.
- CORS methods: `GET`, `POST`, `OPTIONS`, `HEAD`.
- Allow headers: `Content-Type`, `Authorization`, `X-Admin-Token`, `Accept`.
- `supports_credentials=False`.

Localhost regex used in the default origin list:

```
^https?://(?:localhost|127\.0\.0\.1)(?::\d+)?$
```

Free CORS prefixes (`/`, `/health`, `/run`, `/compile`, `/lsp/`, `/ws/`) reflect the request `Origin` even when it is not in `ALLOWED_ORIGINS` so AXIS VPS UI → local pyne preflight succeeds.

Same-origin / no `Origin` header (curl, server-to-server) remains usable under flask-cors behavior.

Blueprints

```
app.register_blueprint(preview_bp)   # url_prefix=/preview
app.register_blueprint(backtest_bp)  # url_prefix=/backtest
app.register_blueprint(lsp_bp)       # url_prefix=/lsp
app.register_blueprint(git_oauth_bp) # /api/git/oauth/*
```

Preview + backtest live in `backend/api/preview.py`. Optional `flask-sock` registers `WS /ws/run` when installed.

Error handlers

Code	Body code	Message
404	`NOT_FOUND`	`Endpoint {path} not found.`
500	`INTERNAL_ERROR`	`Internal server error.`

Both return JSON with `status: "error"`.

Internals

Path	Role
`backend/app.py`	App object, free routes, auth routes, CORS, health
`backend/api/preview.py`	Pro preview + backtest blueprints
`backend/api/lsp_http.py`	Free AXIS LSP-HTTP
`backend/middleware/schemas.py`	Request validation for `/run*` and auth
`backend/middleware/free_limits.py`	Optional bar / script / rate / concurrency caps (`FREE_TIER_LIMITS`)
`backend/requirements.txt`	Flask, flask-cors, numpy, matplotlib, ...

Recommended production: gunicorn/uvicorn-class WSGI with multiple workers; for multi-worker key consistency prefer SQLite/Redis stores over the default in-memory/file singleton assumptions — see auth.

Invariants and edge cases

1. **Import side effects:** creating `app` configures CORS immediately from env.
2. **Body too large** → Flask 413 before route logic (not custom JSON).
3. **Unknown fields** on schema-validated routes → `UNKNOWN_FIELDS` 400 (`/run` , auth); preview routes currently use looser `get_json()` (schemas exist but are not always applied in handlers).
4. **Dev bind default is loopback** — set `HOST=0.0.0.0` only when intentional.

Worked example — smoke test

```
curl -s http://127.0.0.1:5002/health | jq '.status, .features.default_run_mode'
# "healthy"
# "auto"
```

Failure modes

Symptom	Cause
Browser CORS errors	Origin not in `ALLOWED_ORIGINS`
413 on large OHLCV	Exceeded 5 MB — downsample or paginate
Import errors for matplotlib	Missing backend deps — `pip install -r backend/requirements.txt`

See also

- Auth and keys
- Run endpoint
- Docker / CI

AUTH AND KEYS

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE
HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

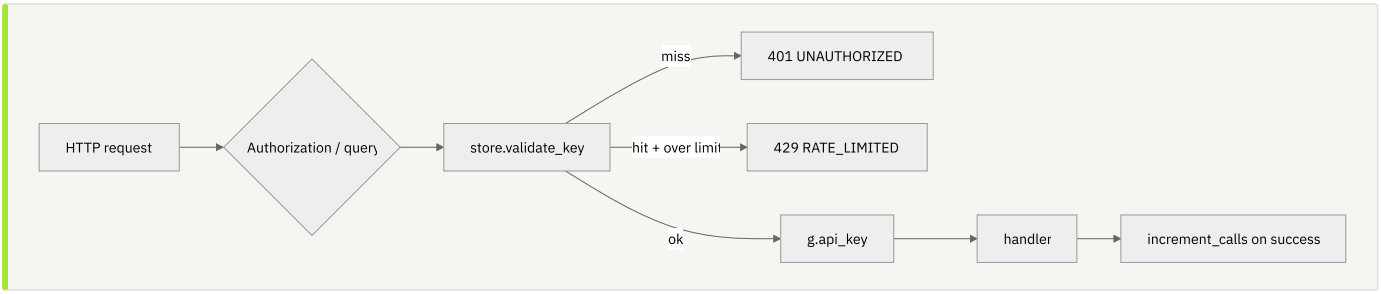
title: "Auth and Keys" description: "API key model, tiers, require_api_key / track_usage decorators, JSON/SQLite/Redis stores, and admin create_key."

AUTH AND KEYS

Abstract

Pro routes authenticate with a **raw API key** presented as `Authorization: Bearer …`, `Authorization: ApiKey …`, or `?api_key=`. Keys are minted as `pyn_` + url-safe secret, stored with a short `key_id` and SHA-256 hash metadata, and tracked against **monthly-ish call limits** by tier. Free `/run` does not require a key.

Conceptual model



Interface surface

Tiers (`_TIER_LIMITS`)

Tier	`calls_limit`
`free`	`0` (treated as unlimited in `is_rate_limited` — limit 0 means “no cap”)
`hobby`	5 000
`pro`	50 000
`team`	200 000
`enterprise`	`inf`

`calls_remaining()` returns `inf` when limit is 0.

Endpoints

`POST /auth/create_key`

Body schema `CREATE_KEY_SCHEMA` : optional `tier` (default `"hobby"`).

Response:

```
{
  "status": "success",
  "api_key": "pyn_…",
  "key_id": "12hexchars",
  "tier": "hobby",
  "message": "Store this API key securely. It will not be shown again."
}
```

Decorated with `@require_admin_token` :

- Requires non-empty `ADMIN_TOKEN` env (unset → **403**, minting disabled)
- Client must send matching `X-Admin-Token` (or `Authorization: Bearer …` / `AdminToken …`)
- Comparison uses `hmac.compare_digest`

`GET /auth/usage`

`@require_api_key` — returns `key_id` , `tier` , and usage block (`calls_used` , `calls_limit` , `calls_remaining` , timestamps).

`POST /auth/validate`

Body: `{ "api_key": "…" }` — validates without incrementing usage. Returns tier info, `active`, `rate_limited`.

Decorators

Decorator	Behavior
`require_api_key`	Resolve key → 401 / 429 or set `g.api_key`
`track_usage`	Wraps `require_api_key`; increments calls when response status < 400 (tuple responses) or always for bare Response
`require_admin_token`	Fail-closed admin gate (`ADMIN_TOKEN` + `X-Admin-Token`)

Client presentation

```
curl -H "Authorization: Bearer $API_KEY" \
  -H "Content-Type: application/json" \
  -d @body.json \
  http://127.0.0.1:5002/preview/chart
```

Internals

`APIKey` dataclass

Fields: `key_id`, `key_hash`, `tier`, `calls_used`, `calls_limit`, `created_at`, `last_used`.

Helpers: `is_active()` (always True today), `is_rate_limited()`, `get_tier_info()`, `increment_calls()`.

Stores

Module	Use
`middleware/auth.py` → `APIKeyStore` + `get_key_store()`	Facade; selects backend via `STORE_BACKEND`
`middleware/key_store_sqlite.py`	Hash-only rows, WAL, multi-worker friendly
`middleware/key_store_redis.py`	Hash-only Redis; multi-replica

`get_key_store()` backends (`STORE_BACKEND`):

Value	Persistence
`json` (default)	File at `API_KEY_STORE` (default `/data/api_keys.json`); **hash-only** object keys. Legacy files keyed by the raw `pyn_…` secret are migrated on load and the raw secret is dropped
`sqlite`	`API_KEY_STORE_SQLITE` (or `.db` beside JSON path); hash-only
`redis`	`REDIS_URL` required; hash-only

Prod compose defaults to `sqlite` on `/data/api_keys.db` so gunicorn workers share state.

Key minting:

```
raw_key = "pyn_" + secrets.token_urlsafe(32)
key_id  = sha256(raw_key).hexdigest()[:12]
key_hash = sha256(raw_key).hexdigest()
```

Schemas

`CREATE_KEY_SCHEMA`, `VALIDATE_KEY_SCHEMA` in `middleware/schemas.py` — strict types, reject unknown fields.

Invariants and edge cases

1. **`/run` is unauthenticated** — rate-limit at reverse proxy if exposed publicly.
2. **Limit 0 means unlimited**, not "zero calls" — free tier limit value is 0 with that semantics.
3. **Usage increments only after handler returns** — failed auth never bills; handler 4xx under `track_usage` skips increment when returned as `(response, status)`.
4. **Revocation** exists on the store API (`revoke_key`); no public HTTP revoke route in `app.py`.
5. **Admin minting is fail-closed** — unset `ADMIN_TOKEN` or wrong `X-Admin-Token` → 403.

Worked example

```
# create (requires ADMIN_TOKEN env + matching header)

curl -s -X POST http://127.0.0.1:5002/auth/create_key \
  -H "X-Admin-Token: $ADMIN_TOKEN" \
  -H 'Content-Type: application/json' \
  -d '{"tier":"hobby"}'

# validate
curl -s -X POST http://127.0.0.1:5002/auth/validate \
  -H 'Content-Type: application/json' \
  -d '{"api_key":"pyn_…"}'
```

Failure modes

Code	Meaning
`UNAUTHORIZED`	Missing/invalid key
`RATE_LIMITED`	`calls_used >= calls_limit` (finite limits)
`FORBIDDEN`	`ADMIN_TOKEN` unset or `X-Admin-Token` mismatch (fail-closed)
`INVALID_TIER`	create_key with unknown tier string
`INVALID_KEY`	validate endpoint miss

See also

- App lifecycle
- Preview
- Security (DevOps)

RUN

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE
HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

title: "POST /run" description: "Free evaluate endpoints: single-script /run and multi-script /run/batch over shared OHLCV."

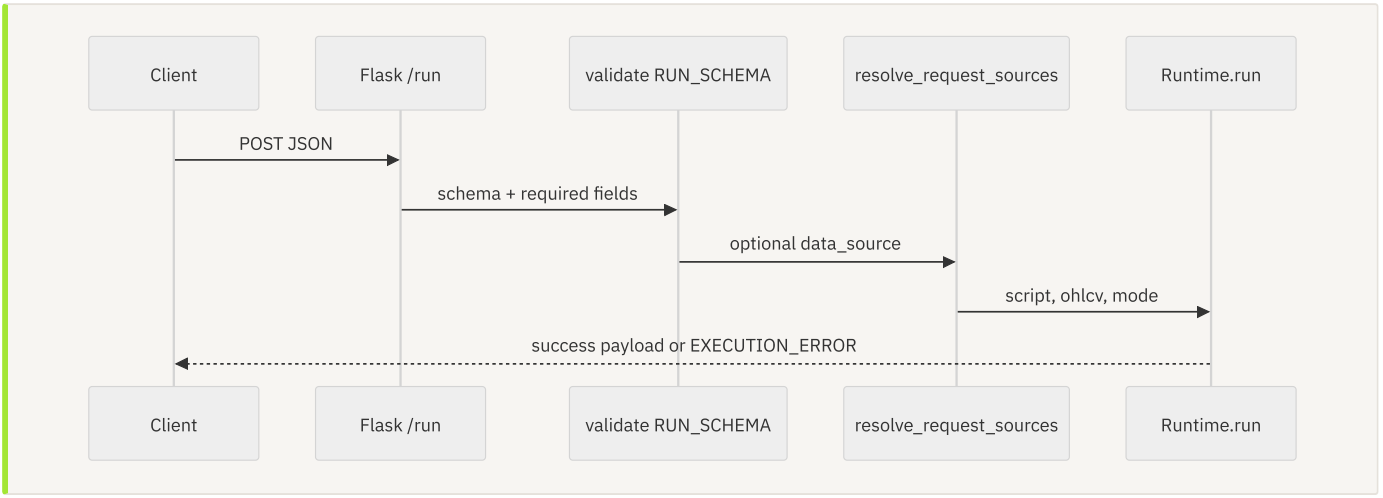
POST `/run` AND `/run/batch`

Abstract

`/run` is the primary free evaluate entry: accept Pine source + bar list, optionally wire `request.*` data sources, execute via package **`pynescript.runtime.Runtime`** (Pro API uses `backend.runtime` re-export), and return plots/series/events/drawings/**alerts**. `/run/batch` runs up to eight scripts on the **same** OHLCV (AXIS multi-indicator), isolating per-script errors so one failure does not discard siblings. Optional **free-tier guards**

(`FREE_TIER_LIMITS` , default off) enforce bar/script caps, rate limit, concurrency, and chart/mock-only data (`backend/middleware/free_limits.py`). Optional **L2 webhooks** POST last-bar (default) `alert()` / `alertcondition()` firings to `webhook_url` or env `ALERT_WEBHOOK_URL` (SSRF-safe).

Conceptual model



Interface surface

`POST /run`

Request (`RUN_SCHEMA`)

Field	Type	Required	Default	Notes
`script`	string	yes		Pine source
`data`	list	yes		OHLCV bar objects
`symbol`	string	no	`"CHART"`	Syminfo / request wiring
`data_source`	string	no	`""`	`mock` / `ccxt` / `yahoo` / ...
`data_options`	object	no	`{}`	Provider options
`mode`	string	no	`"auto"`	`"interpret"` | `"compile"` | `"auto"`
`inputs`	object	no	`{}`	Pine `input.*` overrides by title
`profiler`	bool	no	`false`	Per-line timing on interpret
`webhook_url`	string	no	`""`	L2 alert webhook (overrides `ALERT_WEBHOOK_URL`)
`forward_alerts`	bool	no	`true`	When false, skip outbound webhook
`alert_last_bar`	bool	no	`true`	Webhook only last OHLCV bar firings
`alert_batch`	bool	no	`true`	One batch POST vs per-alert
`libraries`	list	no	`[]`	`[{namespace, name, version, source}]` — AXIS git-publish emulator (max 32)
`timeout_seconds`	number	no	omit / `null`	Optional interpret wall-clock budget. Passed to `Runtime.run` only when set and `> 0` . Omit = no timeout.

Query `?mode=` is accepted when the body omits `mode`.

Unknown fields → `UNKNOWN_FIELDS` 400. Empty script/data still checked after schema with `NO_SCRIPT` / `NO_DATA`. `import` and any `request.` token are compile-ineligible; `mode=auto` falls back to interpret and **keeps** `libraries` on that path.

Bar shape (runtime expectation):

```
{ "time": 0, "open": 1, "high": 1, "low": 1, "close": 1, "volume": 1 }
```

Optional `bid` / `ask` per bar update runtime bid/ask state.

Success response


```
{
  "status": "success",
  "plots": [],
  "series": {},
  "plot_meta": {},
  "events": [],
  "drawings": [],
  "alerts": [],
  "script_id": "16hex",
  "run_id": "16hex",
  "count": 0,
  "mode": "compile",
  "auto_backend": "compile",
  "data_source": "chart",
  "alert_forward": {
    "forwarded": 0,
    "failed": 0,
    "filter": "last_bar",
    "url": "https://hooks.example.com/pine",
    "batch": true
  }
}
```

Field	Meaning
`plots`	Primary series (first plot) — backward compatible list
`series`	Named multi-plot map title → values per bar
`plot_meta`	Per-title color, linewidth, index
`events`	Strategy events with `script_id` / `run_id` stamps
`drawings`	Exported line/label/box registry for AXIS
`alerts`	`alert()` / true `alertcondition()` firings (interpret; empty on compile)
`alert_conditions`	Optional full condition evaluations (when present)
`alert_forward`	Present when a webhook URL was configured; delivery summary
`script_id`	`sha256(source)[:16]`
`run_id`	Per-`Runtime` instance id
`auto_backend`	`compile` | `interpret` when requested `mode` was `auto`
`compile_fallback_reason`	Why `auto` used interpret (eligibility or compile error)
`object_mode` / `compile_cached` / `compile_ms`	Compile diagnostics when that path ran
`nopython_fallback_reason`	Numeric JIT failed; engine re-emitted object mode (still compile)

Errors

HTTP	code	When
400	`MISSING_FIELD` / `INVALID_FIELD` / `UNKNOWN_FIELDS` / `INVALID_BODY`	Schema
400	`NO_SCRIPT` / `NO_DATA`	Empty after defaults
400	`DATA_SOURCE_ERROR`	`resolve_request_sources` failed
400	`WEBHOOK_URL_BLOCKED`	`webhook_url` is private/loopback/metadata (SSRF denylist)
403	`DATA_SOURCE_FORBIDDEN`	Free path rejected live provider (`ccxt` / `yahoo` / …) — only when `FREE_TIER_LIMITS` is on
413	`TOO_MANY_BARS` / `SCRIPT_TOO_LARGE`	Exceeded `FREE_MAX_BARS` (5000) or `FREE_MAX_SCRIPT_CHARS` (256 KiB) — only when `FREE_TIER_LIMITS` is on
429	`RATE_LIMITED` / `TOO_MANY_REQUESTS`	Free IP rate (60 / 60s) or concurrency (4) — only when `FREE_TIER_LIMITS` is on
500	`EXECUTION_ERROR`	Runtime returned `error` key; body may include `error_kind` / `error_type` / `error_bar`

`POST /run/batch`

Request (`RUN_BATCH_SCHEMA`)

Field	Type	Required	Notes
`scripts`	list	yes	strings or `{id, script}` objects
`data`	list	yes	Shared OHLCV
`symbol` / `data_source` / `data_options` / `mode` / `profiler`	optional	same as `/run` (no `inputs`)	
`libraries`	list	no	same AXIS git-publish list as `/run` (max 32; applied to every script)
`timeout_seconds`	number	no	same optional interpret budget as `/run` (applied to every script)
`webhook_url` / `forward_alerts` / `alert_last_bar` / `alert_batch`	optional	same L2 webhook fields as `/run` (applied per script result)	

Hard cap: `RUN_BATCH_MAX_SCRIPTS = 8` → `TOO_MANY_SCRIPTS`.

Response

```
{
  "status": "success" | "partial",
  "results": [ { "id": "…", "status": "success|error", … } ],
  "count": 2,
  "ok": 1,
  "data_source": "chart"
}
```

HTTP **200** for partial success (per-script errors inside `results`). Envelope validation failures still 400.

Internals

```
runtime = Runtime(symbol=str(symbol))
result = runtime.run(script, ohlcv, data_feed=…, data_provider=…, mode=…)
```

See runtime bridge for bar-loop details and compile mode.

Paths: `backend/app.py` (`run_pine_script`, `run_pine_script_batch`, `compile_prewarm`, `WS /ws/run`), `backend/middleware/schemas.py`, `backend/middleware/free_limits.py`, `backend/alert_forwarder.py`.

Invariants and edge cases

1. **No API key** on these routes — free-tier guards apply only when `FREE_TIER_LIMITS` is on (see Pro API usage); protect further at the edge if needed.
2. **Batch isolates exceptions** per script (`try/except` around `runtime.run`).
3. **Whitespace-only `data_source`** coerced to `None` → chart default; free paths reject live providers (ccxt/yahoo/…) when `FREE_TIER_LIMITS` is on.
4. **Compile mode** may omit alert side effects (`alerts: []`); use interpret/`auto` for alerts.
5. Schema rejects extras — clients must not send AXIS-only fields without updating schema.

6. **Webhook delivery is best-effort** — evaluate still returns `status: success` if the hook fails; check `alert_forward`.
7. Structured errors may include `error_kind` (`parse|compile|runtime|data|order|mode`), `error_type`, `error_bar`. `timed_out` is set on the HTTP body when Runtime exceeds `timeout_seconds`.

Worked example

```
curl -s http://127.0.0.1:5002/run \
  -H 'Content-Type: application/json' \
  -d '{
    "script": "//@version=6\nindicator(\"t\")\nplot(close)",
    "data": [
      {"time": 1, "open": 1, "high": 2, "low": 0.5, "close": 1.5, "volume": 10},
      {"time": 2, "open": 1.5, "high": 2.5, "low": 1.0, "close": 2.0, "volume": 12}
    ],
    "symbol": "TEST:DEMO"
  }'
```

Failure modes

Symptom	Cause
`UNKNOWN_FIELDS`	Typo'd property (e.g. `ohlcv` instead of `data`)
Parse errors in message	Invalid Pine — same engine as CLI
Batch `status: partial`	At least one script failed; inspect `results[i].message`

`POST /compile/prewarm`

Free readiness hook (same rate/concurrency gates). Body optional:

```
{ "scripts": ["//@version=6\nindicator(\"x\")\nplot(close)"], "force": false }
```

Caps at 16 sources. Returns `has_numba`, `builtins_warmed`, `scripts_ok` / `scripts_failed`, `prewarm_ms`. Does **not** execute on OHLCV.

`WS /ws/run`

When `flask-sock` is installed: JSON text frames `{type: "run", id, script, data, mode?}`. Reply `{type: "result", …}` or `{type: "error"}`. Ping/pong supported. Same `execute_run_payload` as HTTP `/run`.

The run frame accepts the same optional fields as `POST /run` — `symbol`, `mode`, `data_source`, `data_options`, `inputs`, `profiler`, `libraries`, `timeout_seconds`, `from_bar`, `to_bar` — so published-library imports (`import ns/Name/1`) work over the WebSocket channel exactly as they do over HTTP.

The Pro API container must run gunicorn with the `gthread` (or `sync`) worker: `flask-sock` performs the HTTP→WebSocket upgrade itself. The `geventwebsocket…GeventWebSocketWorker` worker upgrades the same socket a second time, which corrupts the frame stream (clients see close code 1002) while REST keeps working.

Alert webhooks (L2)

```
# Server default (optional)

curl -s http://127.0.0.1:5002/run \
  -H 'Content-Type: application/json' \
  -d '{
    "script": "//@version=6\nindicator(\"a\")\nif bar_index == last_bar_index\n    alert(\"fire\")\
    "mode": "interpret",
    "webhook_url": "https://hooks.example.com/pine",
    "data": [
      {"time": 1, "open": 1, "high": 2, "low": 0.5, "close": 1.5, "volume": 10},
      {"time": 2, "open": 1.5, "high": 2.5, "low": 1.0, "close": 2.0, "volume": 12}
    ]
  }'
```

Full engine rules and batch payload shape: Alerts.

See also

- Alerts
- Contract
- Runtime bridge
- App lifecycle

SESSIONS

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE
HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

title: "Live sessions" description: "POST /session and friends — O(1) forming-bar ticks, O(delta) appends, and response windowing on /run."

LIVE SESSIONS AND RESPONSE WINDOWING

Abstract

A `POST /run` call is **stateless**: it parses the script and walks every bar on every request. That is the right shape for batch and history work, and the wrong shape for a live chart, where the same script is re-evaluated dozens of times a second against a *growing* bar list.

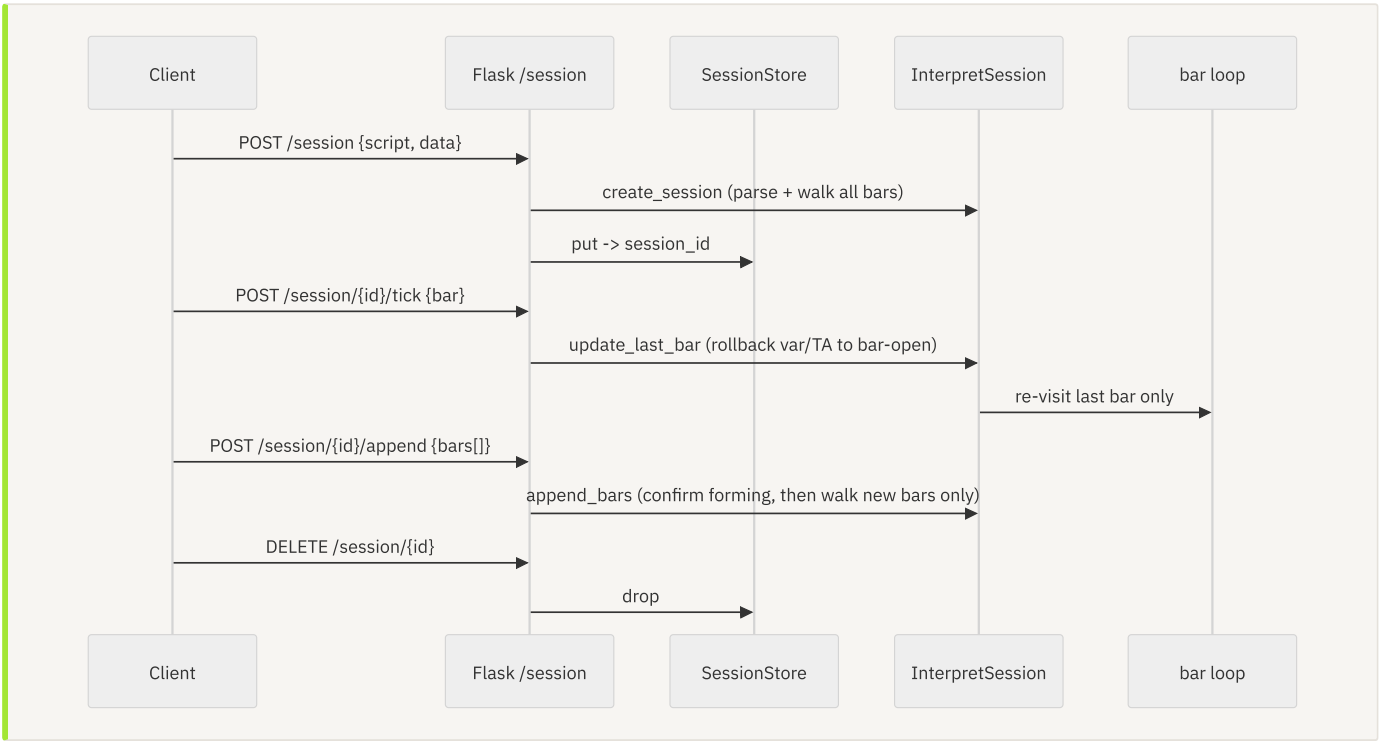
A **session** keeps the parsed tree, its series, and its TA state alive between requests. Advancing it costs work proportional to what actually changed:

Operation	Complexity	Measured (20k bars)
`POST /run` (full re-walk)	O(n)	1,087.8 ms
`POST /session/<id>/tick`	**O(1)**	**0.130 ms** (8,343×)
`POST /session/<id>/append`	**O(delta)**	**0.127 ms** (8,572×)

Session init costs roughly 443 ms (parse + first walk), so a session breaks even after its **first** tick.

Sessions are **interpret only**. The compile path has no resumable evaluator state — the generated bar loop is a single `njit` function over the whole array, with nothing to roll back. `mode: "compile"` on `POST /session` is rejected with `MODE_UNSUPPORTED`.

Conceptual model



Interface surface

`POST /session`

Field	Type	Required	Notes
`script`	string	yes	Pine source
`data`	list	yes	Initial OHLCV bar objects
`symbol`	string	no	Syminfo / request wiring, default `"CHART"`
`inputs`	object	no	`input.*` overrides (interpret only)
`realtime_ticks`	int	no	Re-visits per bar in the realtime window
`realtime_bars` / `realtime_from_bar`	int	no	Realtime window extent
`mode`	string	no	Must be omitted or `"interpret"`

Returns the standard `/run` envelope plus **`session_id`**, `count`, and `mode`.

`POST /session/<session_id>/tick`

Body `{ bar }` — a single OHLCV bar replacing the **last** bar. Bar count and bar index do not advance.

`POST /session/<session_id>/append`

Body `{ bars }` — newly **closed** bars. Previously committed bars stay confirmed; realtime windows do not span append boundaries.

`GET /session/<session_id>`

Small status blob — never the full state:

```
{
  "status": "success",
  "session_id": "f8e72bcf652c4cafbc4ac79b057784ce",
  "bars": 10,
  "idle_seconds": 0.002,
  "ttl_seconds": 900.0
}
```

`DELETE /session/<session_id>`

`{"status": "success", "session_id": "…", "released": true}`. Sessions otherwise go idle and are swept after the TTL.

Tick semantics

`tick` is not an append of one bar. The host:

1. Restores `var` scope and TA windows to the snapshot taken at **bar-open** — `varip` deliberately *persists*, matching Pine's intrabar `varip` semantics.
2. Drops this bar's events, alerts and alert conditions so the re-visit re-emits them.
3. Overwrites the packed column cells **in place** — no append, so bar count is unchanged.
4. Re-visits the tree once and finishes bar plots at the same slot.

Strategy fills are **not** rolled back across ticks. That is the same limitation as the realtime tick loop in `Runtime.run`; strategy series history is overwritten, not appended.

Response windowing

`POST /run`, `/run/batch`, `/session/<id>/tick` and `/session/<id>/append` all accept optional **`from_bar`** / **`to_bar`** fields (inclusive bar indices into `data`; negative counts from the end; either side may be omitted).

Windowing trims the **serialized envelope only**. The bar loop still runs over every bar, so indicator values, strategy state, NaN warm-up, and Pine parity are untouched. Measured on 20k bars against a local Pro API: **1,068.8 KB → 28.0 KB (38× smaller)** for a 500-bar window, with the windowed series verified byte-equal to the full run's tail.

```
curl -s -X POST http://127.0.0.1:5002/run \
  -H 'Content-Type: application/json' \
  -d '{"script":"//@version=6\nindicator(\"x\")\nplot(close)","data":[...],"from_bar":10,"to_bar":1
```

```
{ "count": 20, "window": { "bars": 5, "from_bar": 10, "to_bar": 14, "total_bars": 20 } }
```

`count` still reports bars **evaluated**, not bars serialized. A client seeing `count=20000` with a 500-long series needs the `window` block to tell a window from a short history. The window block is **only present when a window was actually applied** — the unwindowed envelope is unchanged and gains no `window` key.

Events and drawings are filtered to the window when they carry `bar_index`. Objects with only `bar_time` (strategy fills) have no index to match on and are **kept**, so trimming can never silently drop a fill.

Capacity, TTL, and the single-worker constraint

Property	Value
Idle TTL	900 s (15 min), swept at most every 30 s (`SESSION_TTL_SECONDS`)
Capacity	64 concurrent sessions (`SESSION_MAX_SESSIONS`)
Full	`503 SESSIONS_FULL` — back off rather than retrying into a growing pile

Sessions live in a **process-local** `SessionStore`. They only resolve on the single gunicorn worker, which is why `GUNICORN_WORKERS` defaults to `1`. `/health` still advertises the routes (they are part of the capability surface), and `features.live_modes` reports `["every-tick", "bar-close"]` for AXIS capability detection.

Restart recovery

A worker recycle (`--max-requests`) or a container restart empties the process — but every mutation is also journaled to SQLite (`backend/datastore/session_log.py` , `sessions` + `session_bars` tables under `PYNE_SESSION_DB` , default `/data/sessions.db` when `/data` exists, memory-only otherwise). On the first access after a restart the store rebuilds the session with one `create_session` over the logged bars and registers it under its original id, so ticks continue O(1) after that single re-evaluation. Journal writes are best-effort: an unreachable db degrades to memory-only instead of failing the request.

The rebuild is a rebase, not a byte-identical resume: indicators evaluate identically over the same bars, but stateful constructs that never roll back (strategy fills) may differ exactly as if the client had created a new session over the same window. `GET /session/<id>` on a journaled-but-unloaded session answers from the row without rebuilding, and lists `persisted` (journaled rows) next to `sessions` (live objects) in `sessions` stats blocks.

For live data sourced from an exchange rather than the request body, see Data plane — `POST /data/session` binds a session to a **stored** series and advances it from the engine feed.

Failure modes

Symptom	Cause	Fix
`MODE_UNSUPPORTED`	`mode: "compile"` on `/session`	Drop `mode` ; sessions are interpret only
`SESSIONS_FULL`	64 live sessions held	`DELETE` finished ones, or wait for TTL
`404` on tick/append	Session swept (idle TTL)	Create a new session
First tick after restart is slow	Session rebuilt (one full re-evaluation)	Expected; subsequent ticks are O(1) again
`409` on `/data/session/<id>/tick`	Session bound to a different series	Release and rebind
`503 CCXT_UNAVAILABLE`	`ccxt` not installed on the host	`pip install 'pyne[datafeed]'`

Internals (map)

Path	Role
`backend/app.py`	`/session` routes, window application
`backend/middleware/sessions.py`	TTL + capacity `SessionStore` (process-local)
`backend/datastore/session_log.py`	SQLite session journal (restart recovery)
`backend/middleware/window.py`	`apply_window` — serialization-boundary trim
`backend/datastore/live.py`	Session ↔ stored-series binding + sync
`src/pynescript/runtime/host.py`	`InterpretSession`, `append_bars`, `update_last_bar`

See also

- POST /run
- Data plane
- Runtime bridge
- App lifecycle

DATA PLANE

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE
HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

title: "Data plane" description: "Engine-owned OHLCV series store and data-bound live sessions — /data/series* and /data/session*."

DATA PLANE

Abstract

`POST /run` takes its bars **in the request**. That is fine for a one-shot evaluate, but a live chart re-sends the entire history on every tick and asks the exchange for bars the server already has.

The **data plane** moves bars out of the request body and into an engine-owned store:

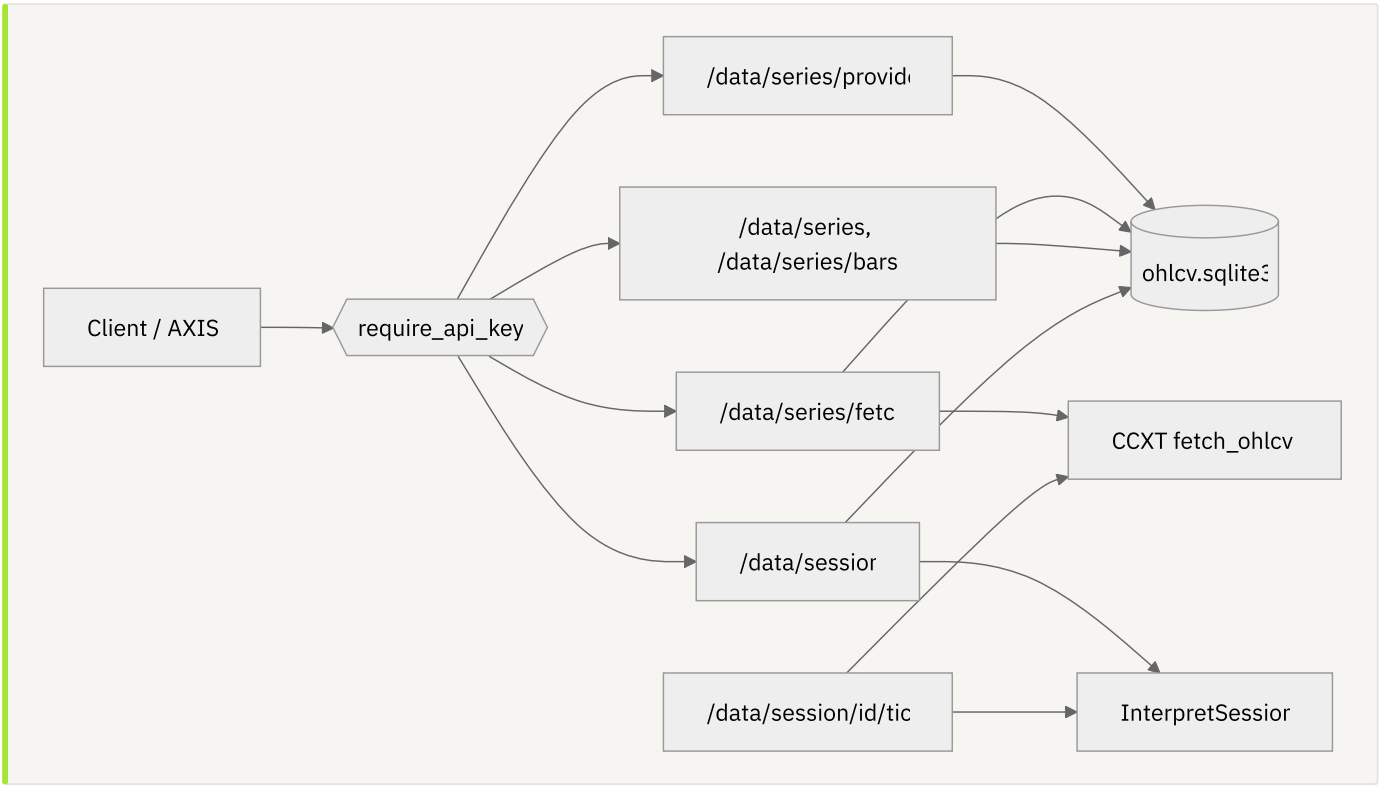
- **`/data/series*`** — a SQLite-backed OHLCV store keyed by a canonical series key. Upload closed bars, fetch history from CCXT, read bounds, read the newest window.

- `/data/session*` — a live session **bound to a stored series**, advanced from the engine feed instead of the client payload.

Bars are keyed by `exchange|market|SYMBOL|interval` — for example `binance|spot|BTC/USDT|1h`. Upserts are keyed by unix-second `time` and the **later write wins**, so a forming bar is re-written in place rather than duplicated.

Implemented in `backend/datastore/series.py` and `backend/api/data_series.py`, without Flask or CCXT in the store itself.

Conceptual model



Series key

Every route takes the same four fields. `market` is optional and defaults to `spot`.

Field	Rule	Example
`exchange`	CCXT id, letters/digits/ `-` / `_`, lowercased	`binance`
`market`	one of the supported markets, default `spot`	`spot`
`symbol`	a pair, uppercased	`BTC/USDT`
`interval`	canonicalized	`1h`

Responses echo the normalized key text, e.g. `binance|spot|BTC/USDT|1h`.

Interface surface

All `/data/*` routes require an **API key** (`Authorization: Bearer …`, `X-API-Key`, or `?api_key=`). They move and persist user market data, so they are not on the free path. Unauthenticated calls return `401 UNAUTHORIZED`.

`GET /data/series`

Stored bounds for one series. Unknown series report count `0` rather than `404`.

```
{ "status": "success", "key": "binance|spot|BTC/USDT|1h",
  "count": 30, "oldest": 1700000000, "newest": 1700104400, "updated_at": 1791968289 }
```

`GET /data/series/bars`

Newest stored bars, **ascending**. `limit` defaults to the store window default and is capped at the store maximum.

```
{ "status": "success", "key": "binance|spot|BTC/USDT|1h", "count": 2,
  "bars": [ { "time": 1700100800, "open": 128.0, "high": 129.0,
              "low": 127.0, "close": 128.5, "volume": 10.0 }, … ] }
```

`POST /data/series/provide`

Upload closed bars. At most **5,000** per request; over that is `413 BATCH_TOO_LARGE` — split the upload into chunks.

```
{ "status": "success", "key": "binance|spot|BTC/USDT|1h",
  "added": 30, "updated": 0, "count": 30, "newest": 1700104400 }
```

`POST /data/series/fetch`

Walk CCXT `fetch_ohlcv` pages forward from `now - history_bars` and persist. `history_bars` defaults to 5,000 and caps at 100,000. Upstream failure is `502 UPSTREAM`; missing `ccxt` is `503 CCXT_UNAVAILABLE`.

```
{ "status": "success", "key": "binance|spot|BTC/USDT|1h",
  "fetched": 5000, "count": 5000, "oldest": …, "newest": … }
```

`POST /data/session`

Build a live interpret session over the stored window and bind it to the series key. Accepts the same script/inputs/realtime fields as live sessions plus `exchange`, `symbol`, `interval`, `market`, and `history_bars`.

Returns the `/session` envelope plus `session_id`, `cache`, and `data`:

```
{ "status": "success", "session_id": "49564cd995514ef48254eb9dd5322e90",
  "count": 20, "cache": "hit",
  "data": { "key": "binance|spot|BTC/USDT|1h", "count": 30, "newest": 1700104400 } }
```

When the store holds fewer than `history_bars` bars it returns `409 DATA_MISS` with `have` and `need` rather than silently evaluating a short history:

```
{ "status": "miss", "code": "DATA_MISS", "have": 0, "need": 50 }
```

`POST /data/session/<session_id>/tick`

Takes `{ exchange, symbol }` only — the series binding is already established. It fetches candles from **two bars before** the stored newest (so a corrected or still-forming bar is picked up), persists closed bars, re-ticks the forming bar with `update_last_bar`, and appends newer bars with `append_bars`.

The response adds `appended`, `forming`, and `unchanged`:

```
{ "status": "success", "session_id": "…", "count": 31,
  "appended": 1, "forming": true, "unchanged": false }
```

`409 SESSION_KEY_MISMATCH` guards a session bound to another series. Upstream failure is `502 UPSTREAM` and **keeps** the session.

Storage

Property	Value
Backend	SQLite (`ohlcv.sqlite3`), single connection
Directory	`PYNE_DATA_DIR`, default `data/`
Identity	`exchange|market|SYMBOL|interval`
Upsert key	unix-second `time`; later write wins

The store writes through a single connection, so the data plane has the same **single-worker** constraint as live sessions (`GUNICORN_WORKERS=1`). Serving it from more than one worker without a shared volume will silently split history.

Failure modes

Symptom	Code / status	Cause	Fix
`401`	`UNAUTHORIZED`	No API key	Send `Authorization: Bearer`
`400`	`INVALID_KEY`	Missing/malformed `exchange` / `symbol`	Use a CCXT id and a pair
`400`	`INVALID_MARKET`	Unsupported market	Use `spot` (default)
`413`	`BATCH_TOO_LARGE`	>5,000 bars in one upload	Chunk the upload
`409`	`DATA_MISS`	Store shorter than `history_bars`	`/data/series/fetch` or `provide` first
`409`	`SESSION_KEY_MISMATCH`	Wrong series for this session	Release and rebind
`502`	`UPSTREAM`	CCXT fetch failed	Check creds / connectivity; session kept
`503`	`CCXT_UNAVAILABLE`	`ccxt` not installed	`pip install 'pyne[datafeed]'`

Internals (map)

Path	Role
`backend/datastore/series.py`	SQLite OHLCV store, `SeriesKey`, upsert/window/status
`backend/datastore/live.py`	Session ↔ series binding + `sync_session`
`backend/api/data_series.py`	`/data/series*` routes, CCXT page walk
`backend/app.py`	`/data/session`, `/data/session/<id>/tick`
`backend/api/datafeed.py`	Shared CCXT provider + candle mapping

See also

- Live sessions
- Auth and keys
- Observability

STATS

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE
HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

title: "GET /stats" description: "Public process-wide runtime statistics — engine, compile, prewarm, deltas, sessions, datafeed and HTTP counters."

`GET /stats`

Abstract

`GET /stats` is a public (no API key) snapshot of what this worker has been doing since it started: request counts and latency, execution-engine runs, compile cache outcomes, prewarm results, live-session delta sizes, and datafeed traffic.

It is **process-local**. Counters reset when the worker restarts — which is honest for the single-worker gunicorn setup live sessions require, but means the numbers describe *this process, since it booted*, not a durable time series. Export them if you need dashboards.

Interface surface

```
GET /stats
```


```
{
  "status": "ok",
  "service": "pynescript-pro-api",
  "version": "1.0.0",
  "timestamp": 1791968289,
  "debug": { "env": "PYNE_DEBUG", "enabled": true, "scopes": [], "level": "DEBUG" },
  "counters":   { "run.total": { "total": 4 }, "run.by_mode": { "total": 2, "by": { "auto": 2 } } },
  "latencies":  { "run.total": { "count": 4, "total_ms": 9251.2, "avg_ms": 2312.8,
                                 "min_ms": 0.36, "max_ms": 9250.3,
                                 "p50_ms": 1.2, "p95_ms": 8974.6 } },
  "gauges":     { "sessions.active": 1, "sessions.capacity": 64, "http.in_flight": 0 },
  "compile_cache": { "source_entries": 2, "ir_entries": 1, "has_numba": true, … },
  "sessions":      { "sessions": 1, "max_sessions": 64, "ttl_seconds": 900.0 },
  "notes": "in-process counters; reset on worker restart"
}
```


Three metric kinds

Kind	Shape	Notes
`counters`	`{total, by?}`	Monotonic. `by` is a dimension breakdown (mode, endpoint, status class, fallback reason).
`latencies`	`{count, total_ms, avg_ms, min_ms, max_ms, p50_ms, p95_ms}`	Percentiles come from a bounded reservoir of the **last 512 samples** per metric, so they describe recent behaviour, not the whole run.
`gauges`	number	Last value — in-flight requests, active sessions, cache entries.

Metric names are dotted (`run.total`, `delta.bars_appended`, `http.request`) so the payload groups naturally.

What is tracked

Namespace	Examples
`http`	`requests`, `by_endpoint`, `by_status_class`, `client_errors`, `server_errors`, `slow_requests`; `in_flight` gauge
`run`	`total`, `bars`, `by_mode`, `mode_resolved`, `auto_backend`, `errors`, `error_kind`, `timed_out`, `compile_cached`, `compile_fallback_reasons`
`interpret`	`bars`, `bar_loop` latency, `errors` by kind, `timeouts`
`compile`	`attempts`, `eligibility`, `host_cache` (hit/miss), `errors`, `run` / `total` latency
`compile_script`	`cache` split by `memory_raw` / `memory_sanitized` / `disk` / `ir_share` / `miss`
`prewarm`	`calls`, `builtin_warm`, `scripts_ok`, `scripts_failed`, total latency
`delta`	`append_calls`, `bars_appended`, `update_last_bar_calls`; `bars_per_append_max` gauge
`session` / `sessions`	engine `created` and `parse`; store `created`, `dropped`, `evicted`, `active`, `capacity`, `active_max`
`datafeed`	`streams`, `exchanges_connected`, `reconnects`, `errors`, `session_binds`, `watch_connections`, `watch_bars`, gateway `ohlcv` / `markets` counters and latency
`provider`	`created`, `fetches`, `bars_fetched`, `fetch_errors`

Relationship to other surfaces

`GET /stats` **complements**, and does not replace:

- the per-run profile in the `/run` envelope (`total_ms`, `parse_ms`, `eval_ms`, per-line rows) — that is client-visible, per-run detail;
- the `compile` block on `GET /health` — that is capability plus cache configuration;
- `GET /auth/usage` — that is business metering for rate limits.

`/health` deliberately stays a cheap liveness probe. It does not absorb these counters.

Cardinality

Dimension values come from closed sets — route rules, `interpret` / `compile` / `auto`, `2xx` – `5xx`, a small set of exchange ids. `compile_fallback_reasons` is truncated to 120 characters for the same reason. The registry is **not** a Prometheus-style label store; do not point a high-cardinality metric at it.

Failure modes

Symptom	Cause	Fix
All counters reset	Worker restarted / redeployed	Expected — counters are in-process
Counters look low behind a load balancer	Multiple workers, each counting its own slice	Aggregate per worker, or run with `GUNICORN_WORKERS=1`
`p95_ms` disagrees with `max_ms`	Percentiles use a 512-sample reservoir	Treat p95 as recent-window; `max_ms` is all-time
`compile_cache` or `sessions` shows `error`	Engine import failed	The rest of the payload still serves — `/stats` never fails the request

Internals (map)

Path	Role
`src/pynescript/stats.py`	Thread-safe counter / latency / gauge registry
`src/pynescript/debug.py`	`PYNE_DEBUG` toggle (reported under `debug`)
`backend/logging_setup.py`	`debug_report()`
`backend/app.py`	`/stats` route, request hooks
`backend/api/datafeed.py`, `backend/middleware/sessions.py`	Datafeed + session counters

See also

- Observability — `PYNE_DEBUG` logging
- App lifecycle
- POST /run

RUNNER

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE
HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

title: "Hosted runner" description: "Optional Flask/container script registry + bar-close scheduler (PYNE_RUNNER). Off by default."

HOSTED RUNNER

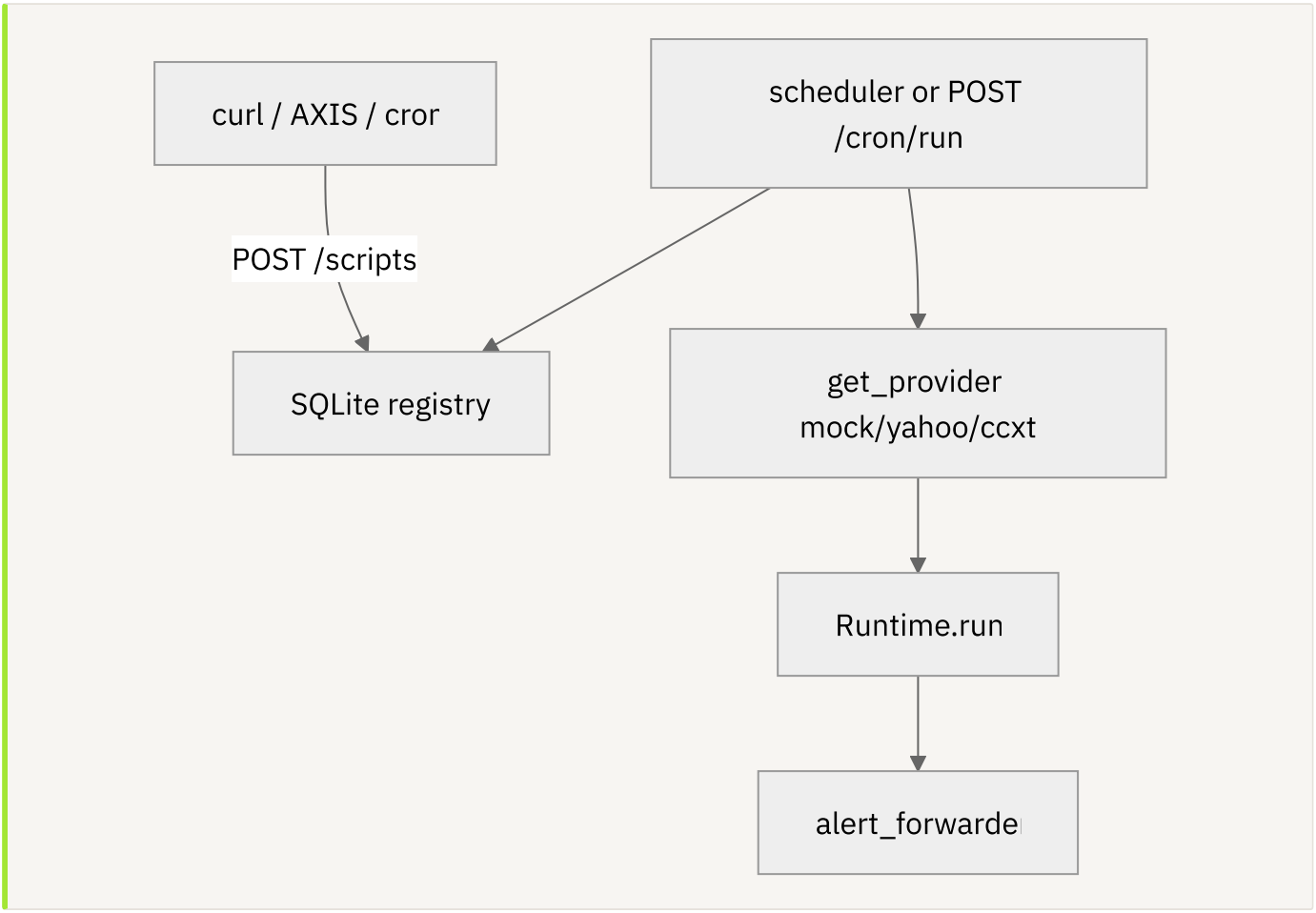
Abstract

The Pro API is request/response: AXIS (or curl) POSTs bars, gets `alerts[]` in that reply, and nothing runs after the client disconnects. **`PYNE_RUNNER`** is an optional Flask-side loop that matches pyne-worker's "upload then keep going" model on **local, Docker, and containers**.

Off unless `PYNE_RUNNER=1` (or `true` / `yes` / `on`). Routes then 404 with `RUNNER_DISABLED`. Health reports `features.script_runner`.

This is **not** HOOX order routing. Alerts leave via the same L2 webhooks as `POST /run`. Strategy events stay on the tick JSON unless you forward them yourself.

Conceptual model



Each tick fetches OHLCV for the script's `symbol` / `timeframe` / `data_source`. If last bar time did not advance, the job is skipped. Otherwise a full `Runtime.run` (same evaluate contract as `/run`) and last-bar alert webhooks.

Interface surface

Environment

Variable	Default	Role
`PYNE_RUNNER`	off	Master switch — HTTP registry + `/cron/run`
`PYNE_RUNNER_SCHEDULER`	off	In-process poll loop (needs `PYNE_RUNNER`)
`PYNE_RUNNER_POLL_SECONDS`	`60`	Poll interval (floor 5s)
`PYNE_RUNNER_DB`	`/data/runner.db` if `/data` exists	SQLite file
`ADMIN_TOKEN`	unset	When set, writes require `X-Admin-Token`

Gunicorn multi-worker: every worker may start a scheduler thread; ticks take a flock on `PYNE_RUNNER_DB.lock`. Prefer `GUNICORN_WORKERS=1` when the scheduler is on.

Routes (`PYNE_RUNNER=1`)

Method	Path	Auth	Role
`POST`	`/scripts`	admin if `ADMIN_TOKEN`	Deploy / replace
`GET`	`/scripts`	none	List (no source)
`GET`	`/scripts/:id`	none	Source + cron state
`DELETE`	`/scripts/:id`	admin if `ADMIN_TOKEN`	Remove
`GET`	`/cron/jobs`	none	Scripts + last tick
`PUT`	`/cron/jobs`	admin if `ADMIN_TOKEN`	`{jobs:[{script_id, enabled}]}`
`POST`	`/cron/run`	admin if `ADMIN_TOKEN`	Tick now (`force` , optional `script_id`)

Deploy body

```
{
  "id": "btc-sma",
  "script": "//@version=6\nindicator(\"t\")\nplot(close)\nalertcondition(close>open,\"up\",\"green\
  "symbol": "BTCUSDT",
  "timeframe": "1d",
  "mode": "interpret",
  "data_source": "mock",
  "period": "6mo",
  "max_bars": 500,
  "enabled": true,
  "webhook_url": "https://hooks.example.com/pine",
  "forward_alerts": true
}
```

`data_source` : `mock` (deterministic, no network), `yahoo` , `ccxt` (`data_options.exchange`), `alphavantage` . Same provider factory as CLI `pyne data` .

Worked examples

Local (no Docker):

```
python -m backend.app   # :5002
pyne runner deploy script.pine --id demo --symbol BTCUSDT --source mock
pyne runner tick --id demo --force
pyne runner list
```

Same HTTP without the CLI:

```
curl -sS -X POST http://127.0.0.1:5002/scripts -H 'Content-Type: application/json' \
  -d '{"id":"demo","script":"//@version=6\nindicator(\"t\")\nplot(close)","data_source":"mock","tim
curl -sS -X POST http://127.0.0.1:5002/cron/run -d '{"force":true}' -H 'Content-Type: application/j
```

Docker:

```
PYNE_RUNNER=1 PYNE_RUNNER_SCHEDULER=1 docker compose up --build api
```

Self-host: set the same env on the API process. SQLite path should sit on a persistent disk (`PYNE_RUNNER_DB=/data/runner.db`).

Cloudflare Container: keep `PYNE_RUNNER=0` in `cf/src/index.ts` unless you want the runner inside the isolate; one gunicorn worker is already the default.

Failure modes

Symptom	Cause	Fix
`RUNNER_DISABLED` 404	Switch off	`PYNE_RUNNER=1`
`FORBIDDEN` 403	`ADMIN_TOKEN` set, header missing	`X-Admin-Token`
`skipped` / `no_new_bar`	Last close unchanged	Wait, or `force: true`
`fetch_failed`	yahoo/ccxt error	Use `mock` or fix `data_options`
Double ticks	Several gunicorn workers	`GUNICORN_WORKERS=1` or rely on flock

See also

- POST /run
- Alerts
- pyne-worker cron — edge equivalent
- Docker
- App lifecycle

OPTIMIZE

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

title: "POST /optimize" description: "Strategy hyperparameter search — N interpret Runtime runs with input.* overrides, TPE/random/grid, holdout or walk-forward."

POST `/optimize`

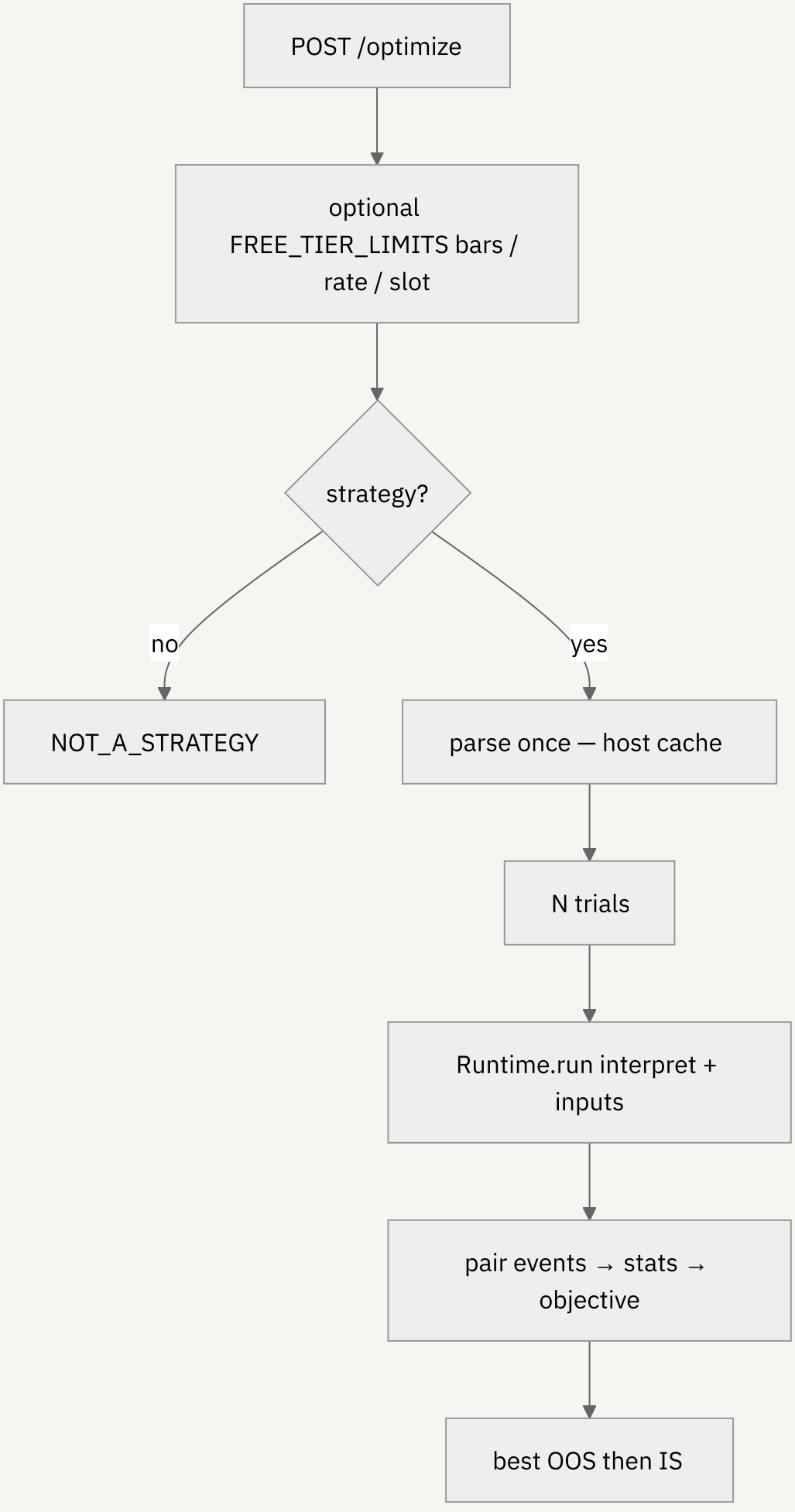
Abstract

`/optimize` searches Pine `input.*` values for a `strategy()` script. It loops the package **Runtime** (same interpret path as `/run`) with different overrides, scores closed trades, and returns the best assignment plus every trial.

This is **not** `/backtest/quick` (that route is a demo SMA simulator). It is **not** a Pine language surface — there is no `optimize.*` builtin.

AXIS Hyperparameter Optimisation calls this when the active engine is the Pro API (`server`). Pyodide / edge engines fall back to a client-side loop.

Conceptual model


POST /optimize
optional FREE_TIER_LIMITS bars / rate / slot
strategy?
no
yes
NOT_A_STRATEGY
parse once — host cache
N trials
Runtime.run interpret + inputs
pair events → stats → objective
best OOS then IS

Request

Field	Type	Required	Default	Notes
`script`	string	yes		Must declare `strategy(`
`data`	list	yes		Same OHLCV objects as `/run`
`space`	object	yes		`{ params: [{ name, kind, min, max, step?, choices? }] }`
`n_trials`	int	no	30	Capped at 200
`sampler`	string	no	`auto`	`auto` | `random` | `tpe` | `grid` (`auto` → random if N<20 else TPE)
`objective`	string	no	`composite`	`net_pnl` | `profit_factor` | `calmar` | `composite`
`validation`	object	no	`{ mode: "holdout" }`	`holdout` | `walk-forward` | `in-sample`
`min_trades`	int	no	5	Trials below this score `-inf`
`seed`	int	no		RNG seed
`symbol`	string	no	`CHART`	
`oos_every_trial`	bool	no	true	Holdout: score every trial on the test slice
`fixed_inputs`	object	no	`{}`	Current `input.*` values for names **not** in `space` (trial params win on overlap)
`libraries`	list	no	`[]`	Same AXIS git-publish list as `/run`

`space.params[].kind`: `int` | `float` | `bool` | `categorical`. Numeric axes **require** `min` and `max`.

`validation` extras: `holdout_frac` (default 0.3), `train_bars` / `test_bars` / `step_bars` for walk-forward.

Estimated engine runs are capped at 400 (`n_trials * folds`).

Success


```
{
  "status": "success",
  "sampler": "tpe",
  "objective": "composite",
  "validation": { "mode": "holdout", "holdout_frac": 0.3, "train_bars": 200, "test_bars": 50, "step
  "n_trials": 30,
  "trials": [
    {
      "index": 0,
      "params": { "Fast": 8 },
      "is_score": 12.4,
      "oos_score": 3.1,
      "is_stats": { "total_pnl": 10, "trades": 8, "max_dd": 0.1 },
      "oos_stats": { "total_pnl": 3, "trades": 6, "max_dd": 0.08 },
      "error": null,
      "engine_runs": 2,
      "ms": 210.4
    }
  ],
  "best_index": 4,
  "best_params": { "Fast": 6 },
  "best_is_score": 15.0,
  "best_oos_score": 4.2,
  "engine_runs": 60,
  "ms": 8421.2,
  "warning": null
}
```


Full event arrays are **not** returned (memory). Re-run the winner via `/run` + `inputs`. Success includes `validation` and per-trial `oos_stats` / `engine_runs` / `ms`.

Errors

HTTP	code	When
400	`MISSING_FIELD` / `INVALID_FIELD` / `UNKNOWN_FIELDS` / `INVALID_BODY`	Schema — empty/ `{}` body, missing required fields, wrong types, extras
400	`NO_SCRIPT` / `NO_DATA`	`script` / `data` present but empty after schema
400	`NOT_A_STRATEGY`	Script is not `strategy()`
400	`INVALID_SPACE`	Missing bounds / empty params
400	`INVALID_VALIDATION`	`validation.mode` not `holdout` | `walk-forward` | `in-sample`
400	`TOO_MANY_RUNS`	Estimated runs exceed 400
400	`GRID_TOO_LARGE`	Grid sampler product exceeds 200 cells
413 / 429	same as `/run`	Free-tier caps — only when `FREE_TIER_LIMITS` is on

Invariants

1. Always **interpret** (input overrides skip compile).
2. Train and test are **bar slices**, not trade filters on a full-sample run.
3. In-sample mode is allowed and **warned**.
4. Ranking prefers **OOS** score when validation is holdout or walk-forward. TPE's good/bad split uses that same score (`oos_score` when the trial has one, otherwise `is_score`).

CLI

```
pyne optimize strat.pine --trials 20 --sampler tpe \
  --space '{"params":[{"name":"Fast","kind":"int","min":3,"max":12}]}'
# walk-forward: --validation walk-forward  (derives train/test from --bars)
# override: --train-bars 80 --test-bars 20 --step-bars 20
```

See also

- POST /run
- Runtime bridge
- AXIS end-user guide: Hyperparameter Optimisation

PREVIEW

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE
HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

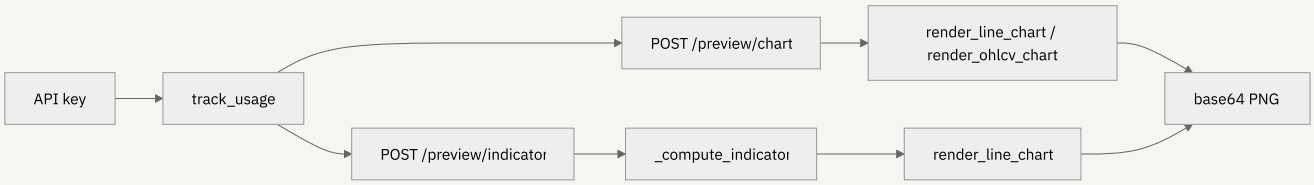
title: "Preview Endpoints" description: "Pro chart and indicator thumbnail routes under /preview with usage tracking."

PREVIEW ENDPOINTS

Abstract

Preview routes render **PNG thumbnails** (base64) for desk UIs and docs. They require an API key via `@track_usage`, accept OHLCV-shaped **objects** (column arrays), and do **not** run the full `Runtime` bar loop for `/preview/chart` (data is plotted directly). `/preview/indicator` evaluates a small set of closed-form `ta.*` expressions in pure Python for the series, then plots.

Conceptual model



Blueprint: `preview_bp` with `url_prefix="/preview"` in `backend/api/preview.py`.

Interface surface

Auth

Both routes: `@track_usage` → must pass `require_api_key` and consume one call on success.

`POST /preview/chart`

Body (documented; schemas exist as `PREVIEW_CHART_SCHEMA`)

```
{
  "script": "optional string",
  "data": {
    "open": [], "high": [], "low": [], "close": [], "volume": []
  },
  "options": {
    "type": "line",
    "color": "#2196F3",
    "show_volume": false,
    "width": 600,
    "height": 300
  }
}
```

Option	Default	Clamp
`type`	`"line"`	`"ohlcv"` selects candlestick renderer
`color`	`#2196F3`	line chart only
`width`	600	max 1200
`height`	300	max 600
`show_volume`	false	twin-axis volume on line chart

Success

```
{
  "status": "success",
  "chart": "<base64 png>",
  "meta": {
    "type": "line",
    "bars": 252,
    "last_value": 0,
    "first_value": 0,
    "change": 0,
    "change_pct": 0,
    "rendered_at": 0
  },
  "tier_info": {}
}
```

Errors

code	When
`NO_DATA`	Missing `data` (both preview routes)
`NO_CLOSE_DATA`	Empty `close` array (chart **and** indicator)
`RENDER_ERROR`	matplotlib / renderer exception (`/preview/chart` only; indicator parse/render exceptions fall back to plotting `close`)

Note: `script` is accepted for API symmetry but **not executed** on this path today — the chart is pure data visualization.

`POST /preview/indicator`

Body

```
{
  "expression": "ta.sma(close, 14)",
  "data": { "close": [/* … */] },
  "options": { "color": "#FF9800", "width": 600, "height": 300 }
}
```

Supported expression prefixes in `_compute_indicator`:

Expression	Series
`ta.sma(…, period)`	SMA of close
`ta.ema(…, period)`	EMA
`ta.rsi(…, period)`	RSI
`ta.macd(…)`	MACD histogram (12/26/9)
other	falls back to raw `close`

Parse failures in the helper fall back to plotting `close`.

Internals

Path	Role
`backend/api/preview.py`	Route handlers + indicator math
`backend/services/chart_renderer.py`	PNG encode
`backend/middleware/auth.py`	`track_usage`

Schemas `PREVIEW_*` in `schemas.py` document intended validation; current handlers use `request.get_json() or {}` directly.

Invariants and edge cases

1. **Data shape differs from `/run`:** preview uses **columnar dicts**; `/run` uses **list of bar dicts**.
2. **Not a full Pine interpreter** for indicator expressions — string prefix parsing only.
3. **Width/height clamps** protect worker memory/CPU.
4. Successful responses include `tier_info` from the authenticated key.

Worked example

```
curl -s http://127.0.0.1:5002/preview/chart \
  -H "Authorization: Bearer $API_KEY" \
  -H 'Content-Type: application/json' \
  -d '{
    "data": {"close": [1,2,3,4,5], "volume": [10,10,10,10,10]},
    "options": {"type": "line", "show_volume": true}
  }' | jq '.meta'
```

Failure modes

Symptom	Cause
401	Missing key
429	Tier limit
Empty-looking PNG	Renderer empty-chart path when all values null

See also

- Chart renderer
- Auth and keys
- Run

BACKTEST

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE
HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

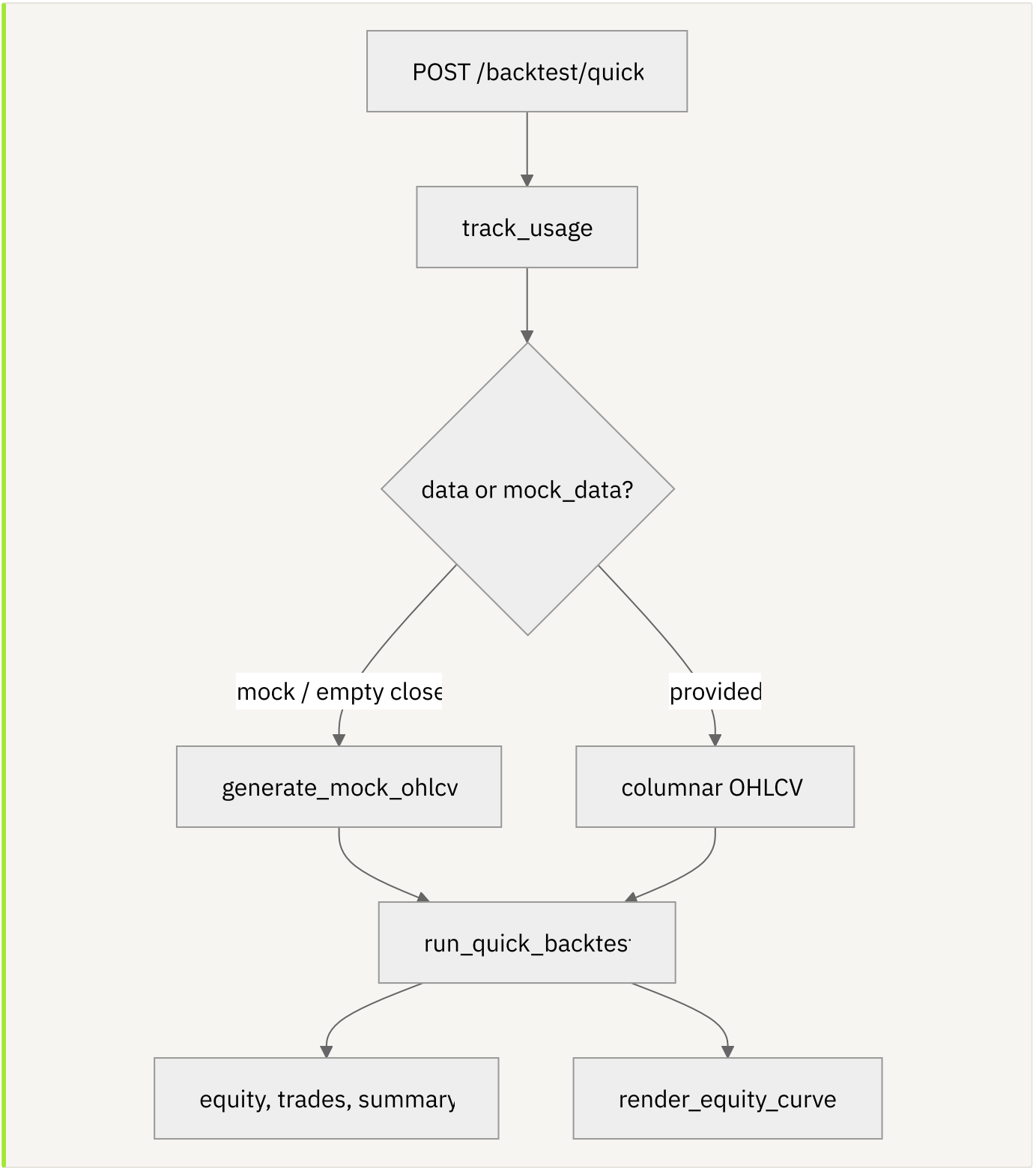
title: "Backtest Endpoint" description: "POST /backtest/quick — usage-tracked strategy metrics, equity curve, and optional mock OHLCV."

BACKTEST ENDPOINT

Abstract

`POST /backtest/quick` is a Pro, usage-tracked route that runs a **simplified** strategy simulation over columnar OHLCV (or generated mock bars), returns trade lists + summary metrics, and embeds a base64 equity-curve PNG. It is optimized for speed/demo UX; it is not a full fidelity substitute for event-aware `Runtime` strategy evaluation on `/run`.

Conceptual model



Blueprint: `backtest_bp`, prefix `/backtest`.

Interface surface

Request

```
{
  "script": "//@version=6\nstrategy(\"s\")…",
  "data": {
    "open": [], "high": [], "low": [], "close": [], "volume": []
  },
  "initial_capital": 10000.0,
  "mock_data": false,
  "mock_bars": 252
}
```

Field	Default	Notes
`script`	required	Empty → `NO_SCRIPT`
`data`	`{}`	Columnar; may be omitted if `mock_data`
`initial_capital`	`10000`	Starting equity
`mock_data`	`false`	Force synthetic bars
`mock_bars`	`252`	Length of mock series

If neither usable `close` nor `mock_data` → `NO_DATA`.

Success


```
{
  "status": "success",
  "result": {
    "equity_curve": [10000, …],
    "trades": [
      {
        "entry_time": 0,
        "entry_price": 0,
        "exit_time": 0,
        "exit_price": 0,
        "direction": "long",
        "pnl": 0,
        "pnl_pct": 0,
        "size": 1
      }
    ],
    "summary": {
      "total_pnl": 0,
      "total_pnl_pct": 0,
      "sharpe_ratio": 0,
      "max_drawdown": 0,
      "max_drawdown_pct": 0,
      "win_rate": 0,
      "profit_factor": 0,
      "total_trades": 0,
      "winning_trades": 0,
      "losing_trades": 0,
      "avg_win": 0,
      "avg_loss": 0
    },
    "equity_chart": "<base64 png>"
  },
  "tier_info": {},
  "meta": {
    "bars": 252,
    "initial_capital": 10000,
    "completed_at": 0
  }
}
```


Errors

code	HTTP	When
`NO_SCRIPT`	400	Empty script
`NO_DATA`	400	No data and not mock
`BACKTEST_ERROR`	500	Exception in simulation
`UNAUTHORIZED` / `RATE_LIMITED`	401 / 429	Auth

Internals

`run_quick_backtest` wraps `run_backtest(..., plot_chart=True)` and returns `BacktestResult.to_dict()` (`backend/services/backtest.py`). `avg_bars_in_trade` exists on the dataclass but is **not** exported in `summary` .

MVP simulation characteristics:

1. Optionally `parse(script)` (errors soft-ignored for MVP path).
2. Precompute long/short entry signals from dual SMA cross (10 vs 20) starting at bar 20.
3. Exit heuristics via RSI-like average and opposite signals / end of series.
4. Equity curve + trade PnL with optional commission/slippage parameters on the lower-level API.
5. Metrics: Sharpe (√252 scaling), max drawdown, win rate, profit factor.
6. Chart via `render_equity_curve`.

`generate_mock_ohlcv(n_bars)` produces synthetic columns for demos.

Path	Role
`backend/api/preview.py`	`quick_backtest` route
`backend/services/backtest.py`	Simulation + metrics
`backend/services/chart_renderer.py`	Equity PNG

Invariants and edge cases

1. **Script content is lightly used** in the MVP sim — do not treat results as broker-accurate for arbitrary strategies. Prefer `/run` events for engine-faithful strategy traces.
2. **Columnar data**, same family as preview, not `/run` bar lists.
3. **Mock path** ignores incomplete user data when `mock_data` or empty close.
4. One usage increment per successful HTTP response under `track_usage`.

Worked example

```
curl -s http://127.0.0.1:5002/backtest/quick \
  -H "Authorization: Bearer $API_KEY" \
  -H 'Content-Type: application/json' \
  -d '{
    "script": "//@version=6\nstrategy(\"demo\")\n// body optional for MVP",
    "mock_data": true,
    "mock_bars": 120,
    "initial_capital": 25000
  }' | jq '.result.summary'
```

Failure modes

Symptom	Cause
Implausible trades vs script logic	MVP signal model, not full evaluator
Empty trade list	No crosses in series / short history
Missing `equity_chart`	Renderer exception swallowed → empty string

See also

- Runtime bridge — faithful strategy events via `/run`

- Chart renderer
- Strategy builtins

CHART RENDERER

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE
HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

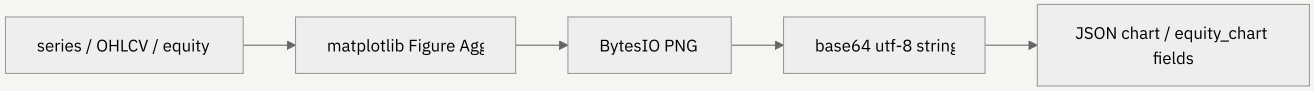
title: "Chart Renderer" description: "Matplotlib Agg PNG services: line charts, OHLCV candles, equity curves, base64 encoding."

CHART RENDERER

Abstract

`backend/services/chart_renderer.py` is a headless plotting façade. It forces `matplotlib` into **Agg**, draws dark-theme charts for desk thumbnails, and returns **base64-encoded PNG** strings suitable for JSON transport. Callers are the preview and backtest routes; the module has no Flask dependency.

Conceptual model



Interface surface

`render_line_chart`

Param	Default	Role
`values`	required	Y series; `None` → NaN gaps
`dates`	`None`	Optional x tick labels (≤ ~6 ticks)
`title`	`"Chart"`	Title string
`color`	`#2196F3`	Line + fill
`height` / `width`	300 / 600	`width` → figsize inches (`width/100`); **`height` is unused** for figsize — line charts use 3.0″ (3.5″ with volume), OHLCV 3.5″, empty 2.5″. dpi 100
`show_volume`	false	Twin axis bars from `ohlcv["volume"]`
`ohlcv`	`None`	Volume source when enabled

Empty / all-null values → `_render_empty_chart` placeholder.

`render_ohlcv_chart`

Candlestick + volume subplot pair from dict keys `open` , `high` , `low` , `close` , `volume` . Green/red body colors (`#4CAF50` / `#F44336`). Shared x-axis. Figsize `(width/100, 3.5)` — the `height` argument is ignored.

`render_equity_curve`

Single series with profit/loss fill relative to zero baseline and a dashed reference at the first equity point. Title fixed `"Equity Curve"` .

Return type

All public renderers → `str` (base64 PNG without data-URI prefix). Clients typically use:

```
{chart}
```

Internals

Detail	Implementation
Backend	`matplotlib.use("Agg")` before pyplot import
Theme	fig `#1E1E1E` , axes `#252526` , muted grids
Cleanup	`plt.close(fig)` after `savefig`
Empty state	Centered “No data available” text

Path	Role
`backend/services/chart_renderer.py`	Renderers
`backend/api/preview.py`	Consumers
`backend/services/backtest.py`	Equity chart hook

Invariants and edge cases

1. **Not interactive** — no GUI backend; safe for servers.
2. **NaNs** break lines at gaps (numpy nan).
3. **Candle body height** uses `+ 1e-9` to keep zero-range bodies visible.
4. **Thread safety:** matplotlib global state is historically process-local; multi-threaded gunicorn workers should avoid concurrent pyplot use on one worker or serialize renders.
5. Exceptions in callers often become HTTP `RENDER_ERROR` or empty equity chart strings.

Worked example

```
from backend.services.chart_renderer import render_line_chart

b64 = render_line_chart([1.0, 1.2, 1.1, 1.4], title="demo", color="#A3E635")
assert isinstance(b64, str) and len(b64) > 100
```

Failure modes

Symptom	Cause
ImportError Agg	Incomplete matplotlib install
Huge base64	High width/height — routes clamp preview sizes
Blank image	Empty input path succeeded with placeholder

See also

- Preview endpoints
- Backtest

RUNTIME BRIDGE

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE
HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

title: "Runtime Bridge" description: "pynescript.runtime.Runtime — package SoT bar-loop evaluate façade, PineSeries context, compile mode, and result packaging."

RUNTIME BRIDGE

Abstract

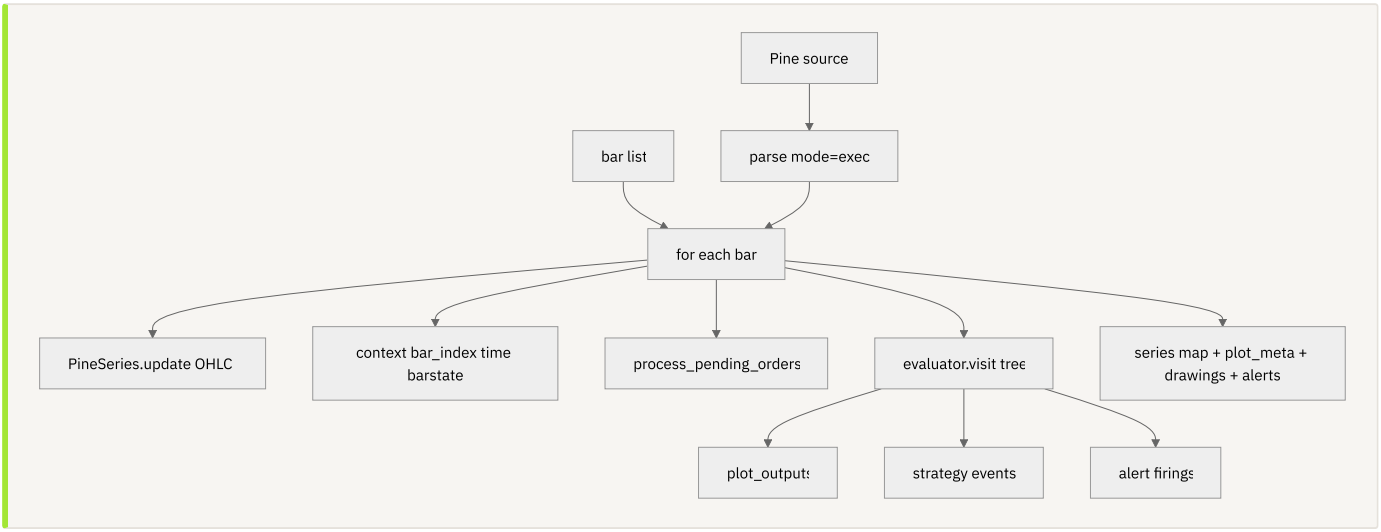
Package Runtime SoT: the bar-loop host, series, and `CustomEvaluator` live under **`pynescript.runtime`** (`src/pynescript/runtime/host.py`). The Pro API still imports `backend.runtime.Runtime`, which **re-exports** the package implementation (`sys.modules` alias) so monorepo and wheel installs share one host.

`Runtime` owns symbol metadata namespaces (`syminfo`, `timeframe`, `barstate`, …), walks OHLCV bars, updates `PineSeries` for OHLC, visits a shared `CustomEvaluator` on the parsed AST each bar, drains strategy events, and packages multi-plot series for AXIS. Compile mode optionally swaps the AST walker for a Numba-backed subset

engine. Optional **`timeout_seconds`** provides a wall-clock circuit breaker on the interpret path (checked every 32 bars) for edge/cron budgets. Flask `/run` and `/run/batch` accept the same field (omit / `null` / `≤ 0` → no timeout).

This is the bridge between Flask handlers and the language core — not a second semantics.

Conceptual model



Interface surface

Construction

```
Runtime(symbol: str = "AAPL", run_id: str | None = None)
```

- Sets `syminfo.tickerid` / `name` / `prefix` (prefix from `EXCHANGE:SYMBOL` form).
- Generates `run_id` as `uuid4().hex[:16]` when omitted.

Helpers: `configure_footprint`, `update_bid_ask`.

`run(source_code, ohlcv_data, data_feed=None, data_provider=None, mode="interpret", …)`

Arg	Role
`source_code`	Pine text
`ohlcv_data`	`list[dict]` with open/high/low/close/time[/volume/bid/ask]
`data_feed` / `data_provider`	`request.*` wiring; auto-resolved from chart bars when unset
`mode`	`"interpret"` (default for direct calls) · `"compile"` · `"auto"` (Pro API schema default)
`timeout_seconds`	Optional wall-clock budget (seconds); interpret path only — partial results + `timed_out` + `error_kind=runtime` when exceeded. Flask `/run` and `/run/batch` accept it (omit / `≤ 0` = no budget)
`libraries`	Optional `[{namespace, name, version, source}]` registered via `register_library_source` before `import ns/Name/ver`. `mode=auto` forwards libs into interpret fallback. Flask `/run` caps at 32
`inputs` / `profiler`	`input.*` overrides (interpret); non-empty `inputs` or `profiler` force interpret under `auto` (`compile_fallback_reason`)
`realtime_*`	Optional host simulation for `varip` multi-tick tests (`realtime_last_bar`, `realtime_ticks`, `realtime_bars`, `realtime_from_bar`)

Interpret path (default)

1. Resolve request sources (non-fatal on failure).
2. `parse(source_code, mode="exec")` — on failure `{ "error": "Parse Error: …" }`.
3. Init `PineSeries` for OHLC + context dict (timeframe daily defaults, barstate, chart colors).
4. Construct `CustomEvaluator(context=…, data_feed=…, data_provider=…)`, reset var declarations + drawing registry + `clear_alerts()`.
5. Per bar:
   - Update series + calendar fields from timestamp
   - Update barstate flags (`isfirst` / `islast` / history confirmed)
   - `process_pending_orders` when available
   - `evaluator.visit(tree)` — on failure `{ "error": "Runtime Error at bar …" }`
   - Lock pine defs after first bar (`_pine_defs_locked`) to avoid multi-dispatch blow-ups
   - Drain strategy events; stamp `script_id` / `run_id`
   - Capture plot outputs
6. Build `series` / `plot_meta` from all plot indices (disambiguate duplicate titles with `_2` suffixes).
7. Export drawings via `DrawingRegistry.export_for_api(bar_times)`.
8. Export alerts via `export_alerts_from_evaluator` (optional `alert_conditions`).

Return keys: `plots`, `series`, `plot_meta`, `events`, `drawings`, `alerts`, `count`, `script_id`, `run_id`, `mode` (+ optional `alert_conditions`, `timed_out`, `error_kind`).

`mode=auto`

1. Non-empty `inputs` → interpret immediately (`compile_fallback_reason = "input.* overrides require interpret path"`).
2. `_compile_eligible`: reject top-level `import` and any `request.` token (cached per source hash).

3. Eligible → `_run_compiled`. On success set `auto_backend=compile`. On compile/runtime error, fall back to interpret and set `compile_fallback_reason`.
4. Ineligible → interpret + `compile_fallback_reason` (e.g. `"import statements not supported in compile path"`).

Value mismatch never triggers a backend switch — that is the parity harness's job.

Compile path

1. Import `pynescript.compiler.engine` — missing → `error_kind=compile`.
2. **Do not** require `has_numba()`. Object-mode scripts (strategy, UDT, drawings) compile to a Python/numpy loop. Missing Numba only fails pure-numeric emit (`CompileNumbaRequiredError`); `auto` caches that failure.
3. Host LRU by raw-source sha256 → `compile_script` on miss.
4. Pack OHLCV to float64 columns **including** `time=` bar-open ms (`_ohlcv_pack_cached`).
5. `compiled.run(opens, highs, lows, closes, volumes, time=times)`.
6. Lift `__drawings` / `__events`; GC compile drawings by declaration caps; merge visual series (bgcolor / plotshape / …) into titled keys.
7. Envelope: `mode: "compile"`, `alerts: []`, `object_mode`, `compile_cached`, `compile_ms`. `generated_code` only when `PYNESCRIPT_RETURN_GENERATED_CODE=1`.

`PineSeries` (`pynescript.runtime.series`)

History deque (default maxlen **1000**, raised by `max_bars_back` / `PYNE_SERIES_MAX`). `PYNE_SERIES_CAP` default **on** (list-length trim). `PYNE_SERIES_RING` default **off** (chronological O(1) lookback when `1`). `series[0]` current / `series[n]` lookback, arithmetic on **current** values, `None` propagates as na-like absence. Negative indices raise — Pine Script™ does not allow them. `backend.series` re-exports the package type.

Context namespaces

Defined on the host: `Syminfo`, `Chartinfo`, `Timeframe`, `Barstate`, `Chart` — attribute names aim at Pine Script™ v5–v6 surface (`timeframe.isdaily`, `syminfo.isin`, …).

Internals

Path	Role
`src/pynescript/runtime/host.py`	Package SoT `Runtime` bar loop
`src/pynescript/runtime/evaluator.py`	`CustomEvaluator` specialization
`src/pynescript/runtime/series.py`	`PineSeries`
`backend/runtime.py`	Compat shim → `pynescript.runtime`
`backend/evaluator.py` / `backend/series.py`	Compat shims
`src/pynescript/ast/helper.py`	`parse`
`src/pynescript/ast/evaluator/**`	Builtin + strategy semantics
`src/pynescript/compiler/engine.py`	Compile mode

Flask wiring: `backend/app.py` constructs a **fresh** `Runtime` per request (and per batch job) so `run_id` and drawing registries do not leak across users.

Invariants and edge cases

1. **Parse once, visit many** — AST is not re-parsed per bar.
2. **Defs locked after bar 0** — performance invariant for large function tables.
3. **Drawing registry reset** at run start — isolation between requests.
4. **Primary `plots` list** is plot index 0 for backward compatibility; AXIS should prefer `series` + `plot_meta`.
5. **Errors are dicts**, not exceptions, for control-flow at the HTTP boundary (`"error" in result`).
6. Compile mode supports a **subset**; `mode=auto` falls back to interpret with `compile_fallback_reason` rather than failing the request.

Worked example

```
from pynescript.runtime import Runtime
# or: from backend.runtime import Runtime  # Pro API monorepo path

rt = Runtime(symbol="NASDAQ:AAPL")
out = rt.run(
    '//@version=6\nindicator("x")\nplot(close, "C")',
    [{"time": i, "open": 1, "high": 2, "low": 0.5, "close": 1.0 + i * 0.01, "volume": 1} for i in r
)
assert "error" not in out
assert out["count"] == 50
assert "C" in out["series"] or "plot_0" in out["series"]
```

Failure modes

Message prefix	Cause
`Parse Error:`	Grammar / syntax
`Runtime Error at bar`	Evaluator exception
`Order fill error at bar`	Broker sim pending orders
`Compile mode requires numba` / `CompileNumbaRequiredError`	Numeric emit without Numba (object-mode scripts still run)
`Compile Error:` / `Compiled Runtime Error:`	Engine path (`error_kind=compile` / `runtime`)
`Script execution timed out`	`timeout_seconds` exceeded (`timed_out`, `error_kind=runtime`)

See also

- Contract
- Run endpoint
- Series and history
- Strategy events

CONTRACT

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE
HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

title: "Evaluate Contract" description: "Shared request/response evaluate contract across Flask Pro API, pyne-worker, and in-process Runtime.run (Python + PyneTS)."

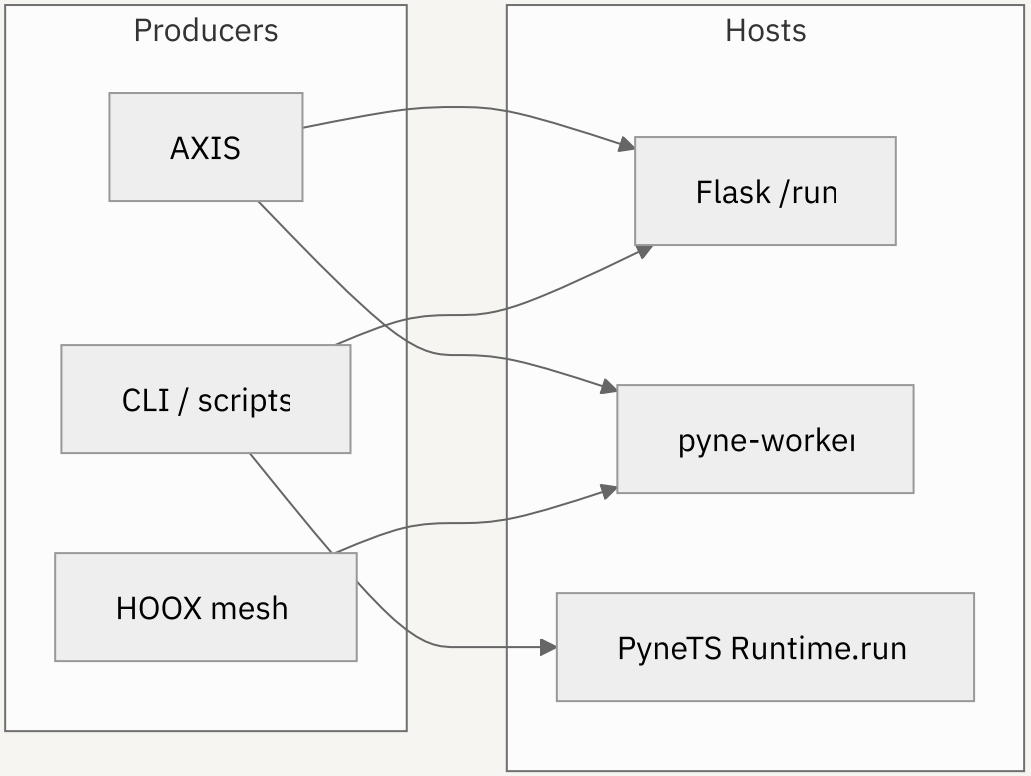
EVALUATE CONTRACT

Abstract

The **evaluate contract** is the JSON dialect that lets AXIS, HOOX, CLI tools, and edge workers call *any* PYNE-compatible host interchangeably. Flask `POST /run` is the reference implementation; Python **pyne-worker** aims for the same HTTP fields. **PyneTS** speaks the in-process form (`RuntimeResult`).

This page freezes the semantic envelope — not transport auth headers.

Conceptual model



Invariant: given the same script + OHLCV + mode, hosts should agree on plot series and strategy events within known parity limits (see numerical validation and PyneTS parity). AXIS engines call Python hosts today, not PyneTS.

Interface surface

Request (single evaluate)

Field	Type	Required	Description
`script`	string	yes	Full Pine source
`data`	`Bar[]`	yes	Chronological OHLCV (`ohlcv` alias on pyne-worker)
`symbol`	string	no	Default `CHART` / host-specific
`mode`	`"interpret"` | `"compile"` | `"auto"`	no	`Runtime.run` default when omitted: `PYNE_RUNTIME_MODE` or `interpret` . Flask `RUN_SCHEMA` default **`auto`**
`inputs`	object	no	`input.*` overrides keyed by title (forces interpret under `auto`)
`libraries`	`{namespace, name, version, source}[]`	no	In-process `import ns/Name/ver` . Flask `/run` and `/run/batch` accept this (max 32).
`profiler`	bool	no	Per-line interpret timings (forces interpret)
`timeout_seconds`	number	no	Optional wall-clock interpret budget (checked every 32 bars; sets `timed_out`). Flask `/run` and `/run/batch` accept it; omit / `null` / `≤ 0` → no timeout.
`webhook_url`	string	no	Flask L2 alert webhook (overrides `ALERT_WEBHOOK_URL`)
`data_source`	string	no	Optional external history provider id (free paths: chart / mock only)
`data_options`	object	no	Provider configuration

`Bar`

Field	Type	Required	Notes
`time`	number	recommended	Unix ms preferred
`open`	number	yes	
`high`	number	yes	
`low`	number	yes	
`close`	number	yes	
`volume`	number	no	Default host-dependent
`bid` / `ask`	number	no	Runtime bid/ask hooks

Response (success)

Field	Type	Description
`status`	`"success"`	Flask sets this; workers may omit and use HTTP 200 only — **prefer including it**
`plots`	`(number|null)[]`	Primary series (plot 0)
`series`	`Record<string, (number|null)[]>`	Named multi-series
`plot_meta`	`Record<string, PlotMeta>`	Display metadata
`events`	`StrategyEvent[]`	Strategy / trade-like events
`drawings`	`Drawing[]`	line/label/box export
`alerts`	`AlertEvent[]`	`alert()` / true `alertcondition()` firings (interpret and object-mode compile; nopython stays `[]`)
`alert_conditions`	object[]	optional condition evaluations
`alert_forward`	object	optional L2 webhook delivery summary
`count`	number	Bars evaluated
`script_id`	string	Stable hash prefix of source
`run_id`	string	Per-invocation id
`mode`	string	Effective mode (`interpret` | `compile`)
`auto_backend`	string	`auto` only: which path ran
`compile_fallback_reason`	string	`auto` only: why compile was skipped or failed
`object_mode`	bool	Compile path: pure-Python bar loop (no njit)
`timed_out`	bool	Interpret budget exceeded (`Runtime.run(timeout_seconds=…)`)
`data_source`	string	Flask echo of resolved source label

`PlotMeta` (Flask)

```
{
  "title": "C",
  "color": "#2196F3",
  "linewidth": 1,
  "index": 0,
  "kind": "plot"
}
```

`kind` is `plot` | `hline` | `fill` | `bgcolor` | `plotshape` | `plotchar` | `plotarrow`. Fill bands add `plot1` / `plot2` title refs; hline may add `price`. Extra style keys (`style`, `linestyle`, `location`, `text`, `char`, `size`) appear when the interpret collector has them.

`StrategyEvent` (minimum)

Hosts should preserve:

```
{
  "type": "string",
  "bar_index": 0,
  "script_id": "…",
  "run_id": "…"
}
```

Additional fields follow evaluator `to_dict()` / TS port parity — treat unknown keys as forward-compatible.

`AlertEvent` (minimum)

```
{
  "message": "string",
  "freq": "once_per_bar",
  "bar_index": 0,
  "time": 0,
  "source": "alert",
  "script_id": "…",
  "run_id": "…"
}
```

See Alerts for frequency rules and webhooks.

Response (failure)

Flask `/run`:

```
{
  "status": "error",
  "code": "EXECUTION_ERROR",
  "message": "Runtime Error at bar …",
  "error_kind": "runtime"
}
```

Optional `error_type`, `error_bar`, `logs`, `profile`, `meta` pass through when `Runtime` set them. `error_kind` is `parse` | `compile` | `runtime` | `data` | `order` | `mode`.

Workers may use equivalent `{ error: string }` or HTTP 4xx/5xx with message body. Clients should accept:

1. Envelope `status === "error"` with `message`, or
2. Presence of top-level `error` string (raw `Runtime.run` shape).

Batch evaluate (Flask extension)

`POST /run/batch`:

Request: `scripts: (string | {id, script})[]` (max 8) + shared `data` + optional `symbol` / `data_source` / `data_options` / `mode` / `profiler` / `libraries` / `timeout_seconds` / webhook fields. **Not** on the batch schema: `inputs`.

Response:

```
{
  "status": "success" | "partial",
  "results": [ /* per-script success or error object with id */ ],
  "count": 0,
  "ok": 0,
  "data_source": "chart"
}
```

Workers may implement batch as N single evaluates; AXIS should not assume batch exists on every host.

Internals — reference mapping

Contract field	Flask source
`plots` / `series` / `plot_meta`	`pynescript.runtime.host.Runtime.run` packing loop
`events`	`evaluator._strategy_state.drain_events()` → `to_dict()` (compile: `__events`)
`drawings`	`DrawingRegistry.export_for_api` (compile: `__drawings`)
`alerts`	`export_alerts_from_evaluator` (interpret); object-mode compile `__alerts` (nopython `[]`)
`alert_forward`	`backend/alert_forwarder.maybe_forward_run_alerts`
`script_id`	`sha256(source)[:16]`
`run_id`	`Runtime._run_id`
`compile_fallback_reason`	`Runtime._run_auto`
`timed_out`	Interpret bar loop vs `timeout_seconds` (Flask `/run` and `/run/batch` when the field is set and `> 0`)

Schema enforcement on Flask free tier: `RUN_SCHEMA` / `RUN_BATCH_SCHEMA` in `backend/middleware/schemas.py` (strict, reject extras). Webhook fields: `webhook_url`, `forward_alerts`, `alert_last_bar`, `alert_batch`. `libraries` and `timeout_seconds` are on both `/run` and `/run/batch`.

Tests: `tests/test_backend.py` (`test_run_success`, `test_run_exports_alerts`, webhook cases), `tests/test_alert_forwarder.py`.

Divergences to remember

Surface	Divergence
Preview / backtest	Columnar `data` dict, not `Bar[]`; not the evaluate contract
Compile mode	Extra keys `object_mode`, `compile_cached`, `compile_ms`, `nopython_fallback_reason`. `generated_code` only if `PYNESCRIPT_RETURN_GENERATED_CODE=1`
`timeout_seconds`	In-process `Runtime.run`, edge workers, and Flask `/run` / `/run/batch` (optional; omit = no budget)
Health	Flask `GET /` and `GET /health` (CORS-free); not part of evaluate
Auth	Flask free `/run` open (bar/rate/concurrency caps); workers may require HOOX mTLS / API gateways
Error codes	Flask uses `code` enums; workers should map to stable strings when possible

Invariants and edge cases

1. **Bar order is chronological ascending** — hosts do not sort for you.
2. `na` **/ missing** appear as JSON `null` in series arrays.
3. **Duplicate plot titles** get suffixes (`_2` , …) on Flask; clients should key on returned map keys, not assumed titles.
4. **Idempotent script_id** for identical source text across hosts.
5. **run_id is not stable** across retries — use for log correlation only.
6. Parity is best-effort on floating edges; pin fixtures when testing hosts against each other.

Worked example — host-agnostic client sketch

```
type EvaluateRequest = {
  script: string;
  data: Array<{
    time: number;
    open: number;
    high: number;
    low: number;
    close: number;
    volume?: number;
  }>;
  symbol?: string;
  mode?: "interpret" | "compile" | "auto";
  libraries?: Array<{
    namespace: string;
    name: string;
    version: number;
    source: string;
  }>;
};

type EvaluateSuccess = {
  plots: Array<number | null>;
  series: Record<string, Array<number | null>>;
  plot_meta: Record<string, Record<string, unknown>>;
  events: Array<Record<string, unknown>>;
  drawings: Array<Record<string, unknown>>;
  alerts: Array<Record<string, unknown>>;
  count: number;
  script_id: string;
  run_id: string;
  mode: string;
  auto_backend?: "compile" | "interpret";
  compile_fallback_reason?: string;
};
```

POST that body to Flask `/run` or the worker's evaluate route; branch on `status` / `error` .

Failure modes

Client bug	Symptom
Sending columnar preview data to `/run`	Schema `data must be a list`
Assuming batch on worker	404 / unknown field
Ignoring `series` and only reading `plots`	Multi-plot scripts look single-series

See also

- Run endpoint
- Runtime bridge
- pyne-worker
- pyne-agent-worker
- Pro API usage (end user)
- Events